

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук

Образовательная программа «Программная инженерия»

УДК004.8

УТВЕРЖДАЮ
Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель департамента
программной инженерии

«__» _____ 2026 г. Н.А. Павлов

Выпускная квалификационная работа

на тему “Бенчмарк для оценки эффекта Reversal Curse в LLM”

по направлению подготовки 09.03.04 «Программная инженерия»

Научный руководитель

Старший преподаватель
департамента ПИ ФКН НИУ ВШЭ

Должность, место работы

Павлов Н. А.

И.О. Фамилия

12.05.2026

Подпись, Дата

Выполнил

студент группы БПИ227
4 курса бакалавриата
образовательной программы
«Программная инженерия»

Бородин М. В.

И.О. Фамилия



11.05.2026

Подпись, Дата

Соруководитель / Консультант

Руководитель группы ООО “Яндекс”

Должность, место работы

Ляликова И. А.

И.О. Фамилия



11.05.2026

Подпись, Дата

РЕФЕРАТ

Большие языковые модели стали важным компонентом современных интеллектуальных систем: поисковых и голосовых ассистентов, чат-ботов, систем поддержки пользователей и внутренних аналитических инструментов. При этом надёжность фактических знаний таких моделей остаётся ограниченной. Один из показательных эффектов - reversal curse, или «проклятие обращения»: модель может корректно отвечать на прямой факт вида «А связано с В», но давать ошибку при обращённом запросе «какой А связан с В». На практике эта проблема усиливается в многокритериальных запросах, где необходимо одновременно удерживать несколько условий, комбинировать факты из разных отношений и не подменять одно ограничение похожим, но неверным.

Настоящая выпускная квалификационная работа посвящена разработке русскоязычного исследовательского бенчмарка для оценки способности LLM отвечать на сложные multi-hop и многокритериальные запросы. В качестве основного источника истины используется Wikidata: для запросов автоматически строятся формальные ограничения, SPARQL-запросы, эталонные ответы в виде QID и русских названий, а также естественно-языковые формулировки запросов. Дополнительно исследуются внешние датасеты RuBQ, MKQA, KQA Pro, Mintaka и 2WikiMultiHopQA как потенциальные источники сложных логических паттернов и вспомогательных срезов.

Основная гипотеза работы заключается в том, что для проверки таких ошибок недостаточно обычного набора одношаговых вопросов. Нужен корпус, в котором есть формально проверяемый основной слой, дополнительные сложные запросы из внешних источников и отдельные примеры, где корректного ответа не существует. Такая структура должна позволить увидеть ошибки, которые часто остаются скрытыми в стандартных QA-бенчмарках: потерю одного из условий, слабое объединение нескольких фактов, асимметрию знания при обращении связи и галлюцинации в ситуациях, где множество правильных ответов пусто. Поэтому в работе датасет разделён на несколько срезов: основной сложный Wikidata-слой, простой контрольный срез, внешний слой сложных одноответных запросов и zero-gold срез для проверки поведения модели при отсутствии ответа.

В финальной версии проекта к базовой генерации были добавлены два инженерных этапа, без которых оценка была бы менее устойчивой. Во-первых, уже собранные gold-ответы Wikidata-слоя дополнительно обогащаются: для них уточняются русские названия, варианты написания и служебные признаки, которые помогают сопоставлять ответ модели с эталоном. Во-вторых, разные срезы датасета приводятся к единому Wikidata-like формату. При объединении сохраняются источник запроса, тип answerability, ограничения и gold-поля. За счёт этого внешний слой не теряет происхождение и не смешивается с основным корпусом как однородная масса, а остаётся отдельной аналитической частью бенчмарка.

Отдельный слой в работе связан с околосинтетическими критериями. Это редкие регистрационные или идентификационные признаки, которые добавляются в часть запросов намеренно. Их роль состоит в том, чтобы сделать задачу менее похожей на проверку заученной энциклопедической памяти. Если условие встречается редко, модели труднее угадать ответ только по параметрам: ей нужно найти факт во внешнем источнике и затем совместить его с остальными ограничениями — жанром, страной, периодом, типом объекта и другими признаками.

Экспериментальные прогоны и качественный анализ ответов моделей показали, что бенчмарк выявляет не только факт ошибки, но и её тип. На простых запросах модели могут выглядеть устойчивыми, однако на многокритериальных списочных запросах чаще проявляются пропуск отдельных условий, неполные списки, подмена похожих сущностей, некорректное обращение фактической связи, галлюцинации и ложные отказы от ответа. Для моделей без поиска характерны отказы вида «не удалось найти информацию» или попытки ответить из параметрической памяти, тогда как поисковые и reasoning-системы чаще находят релевантные сущности, но всё ещё могут терять часть constraints после retrieval-шага.

Ключевые слова: большие языковые модели, LLM, reversal curse, проклятие обращения, multi-hop question answering, многокритериальные запросы, Wikidata, KQA Pro, RuBQ, МКQA, Mintaka, 2WikiMultiHopQA, LLM-as-a-Judge, русскоязычный бенчмарк.

ABSTRACT

Large language models have become a central component of modern intelligent systems, including search assistants, voice assistants, chatbots, customer-support systems, and internal analytical tools. However, the reliability of factual knowledge in such models remains limited. One representative failure mode is the reversal curse: a model may correctly answer a direct fact of the form “A is related to B” while failing on the inverse query “which A is related to B”. In real applications, this problem becomes even more visible in multi-criteria requests, where the model must simultaneously satisfy several constraints, combine facts from different relations, and avoid replacing the requested conditions with similar but incorrect ones.

This thesis project develops a Russian-language research benchmark for evaluating the ability of LLMs to answer complex multi-hop and multi-criteria queries. Wikidata is used as the main source of truth: the pipeline automatically builds formal constraints, SPARQL queries, gold answers represented by QIDs and Russian labels, and natural-language query formulations. In addition, external datasets such as RuBQ, MKQA, KQA Pro, Mintaka, and 2WikiMultiHopQA are analyzed as potential sources of complex reasoning patterns and auxiliary benchmark splits. The main research hypothesis is that a controlled Wikidata core, augmented with an external complex layer and a separate zero-gold split, can reveal systematic LLM errors that are poorly captured by standard one-hop QA benchmarks: constraint omission, weak compositional reasoning, asymmetry of factual knowledge, and hallucinations when the correct answer set is empty. To test this hypothesis, the work proposes a multi-layer dataset architecture: a hard Wikidata core, an easy/shallow control split, an auxiliary complex single-answer split based mostly on KQA Pro, and a zero-gold hallucination-probe split.

In the final version of the project, the core pipeline was extended with two important engineering stages. First, existing Wikidata gold answers are enriched with Russian labels, alternative names, aliases and auxiliary fields that make answer matching and manual inspection more robust. Second, several dataset slices from different sources are merged into a unified Wikidata-like schema while preserving the origin, answerability type, constraints and gold fields of each example. This makes it possible to evaluate the benchmark as a single artifact while still analyzing every source-specific split separately.

Special attention is paid to semi-synthetic criteria: rare registry-like or identifier-like constraints deliberately added to part of the queries. They reduce the probability that the model can answer from parametric memory alone and make it possible to evaluate retrieval-based behavior: finding a rare fact via search and combining it with the remaining constraints.

Experimental runs and qualitative analysis of model answers show that the benchmark detects not only incorrect answers, but also interpretable error types. Simple QA examples may hide weaknesses that become visible in list-style multi-constraint queries: omitted constraints, incomplete answer lists, entity substitution, failed inverse retrieval, hallucinated objects, and false refusals. Models without search tend to either refuse difficult queries or answer from

unreliable parametric memory, while retrieval-augmented and reasoning systems handle rare facts better but can still lose constraints after retrieval.

Keywords: large language models, LLM, reversal curse, multi-hop question answering, multi-criteria queries, Wikidata, KQA Pro, RuBQ, MKQA, Mintaka, 2WikiMultiHopQA, LLM-as-a-Judge, Russian benchmark.

СОДЕРЖАНИЕ

РЕФЕРАТ	2
ABSTRACT.....	4
СОДЕРЖАНИЕ	6
ВВЕДЕНИЕ	10
ГЛАВА 1. ТЕКУЩЕЕ СОСТОЯНИЕ ОБЛАСТИ ПРОБЛЕМЫ	12
1.1. Проклятие обращения и фактические знания LLM.....	12
1.2. Multi-hop и многокритериальные запросы как класс запросов	13
1.3. Существующие датасеты и ограничения текущей практики	13
1.4. Постановка исследовательской проблемы и гипотезы	14
1.5. Выводы по главе 1	15
ГЛАВА 2. ОПИСАНИЕ РЕШЕНИЯ	16
2.1. Требования к бенчмарку и общий метод исследования	16
2.2. Многоуровневая архитектура датасета.....	16
2.3. Wikidata-core: генерация, ограничения, gold-ответы и уровни сложности.....	17
2.4. Околосинтетические критерии и проверка поискового поведения	19
2.5. Анализ внешних источников и формирование слоя сложных запросов	20
2.5.1. Общая методика анализа внешних датасетов	21
2.5.2. RuBQ и МКQA: русские формулировки и ограничения для core-слоя.....	21
2.5.3. KQA Pro: основной внешний донор сложных reasoning-паттернов	22
2.5.4. Mintaka: формальная multi-hop метка и проблема shallow bridge	24
2.5.5. 2WikiMultiHopQA: большой объём и узкая диагностическая польза	25
2.5.6. Итоговая роль внешнего слоя сложных запросов	26
2.6. Срез запросов провокаций на галлюцинации	27
2.7. Формат записи, репозиторий и воспроизводимость	28
2.8. Выводы по главе 2	29
ГЛАВА 3. ЭКСПЕРИМЕНТАЛЬНАЯ ВАЛИДАЦИЯ РЕШЕНИЯ	31
3.1. Валидация источников данных	31
3.2. Проектная структура итогового датасета	31
3.3. Инфраструктурный пайплайн инференса.....	35
3.4. Перефразирование, перевод и подготовка входов моделей	36
3.5. Состав моделей для инференса	37
3.6. Метрики, реворды и LLM-as-a-Judge.....	38
3.7. Санити-чек, ручная валидация и ассессорская разметка	40
3.8. Ограничения, риски и направления развития	41
3.9. Выводы по главе 3	42

ЗАКЛЮЧЕНИЕ.....	49
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	51
ПРИЛОЖЕНИЕ А. СТРУКТУРА ОДНОЙ ЗАПИСИ БЕНЧМАРКА	53
ПРИЛОЖЕНИЕ Б. ПРОМПТЫ ПЕРЕФРАЗИРОВАНИЯ, ПЕРЕВОДА И LLM-AS-A-JUDGE.....	54
Б.1. Промпт перефразирования русскоязычных запросов.....	54
Б.2. Промпт перевода внешних запросов на русский язык	55
Б.3. Промпт LLM-as-a-Judge	55
ПРИЛОЖЕНИЕ В. СВЯЗЬ ГИПОТЕЗЫ С МЕТРИКАМИ И РЕВОРДАМИ	57

ИСПОЛЬЗУЕМЫЕ ОПРЕДЕЛЕНИЯ И ТЕРМИНЫ

В работе используются как общепринятые ML-термины, так и русские обозначения для длинных англоязычных конструкций. Термины LLM, retrieval, baseline, prompt, SPARQL и QID оставлены без полного перевода, так как они являются стандартными для области. Длинные англоязычные кальки ниже заменяются русскими обозначениями.

Термин	Используемое русское обозначение
Reversal Curse	проклятие обращения
multi-hop запрос	многошаговый запрос
multi-constraint запрос	многокритериальный запрос
Wikidata-core	основной Wikidata-слой
external complex layer	внешний слой сложных запросов
truth layer	слой источника истины / формальный слой проверки
benchmark-core	основной слой бенчмарка
split	срез датасета
gold-ответ	эталонный ответ
zero-gold запрос	запрос без корректного ответа
hallucination probe	срез для проверки галлюцинаций
LLM-as-a-Judge	модель-судья / оценка с помощью модели-судьи

Большая языковая модель (LLM) - модель машинного обучения, обычно основанная на архитектуре трансформера и обученная на больших текстовых корпусах. LLM способна генерировать текст, отвечать на вопросы, следовать инструкциям и выполнять рассуждения в естественном языке.

Фактические знания LLM - сведения о сущностях, свойствах и отношениях, которые модель неявно усвоила в параметрах во время обучения. В отличие от базы знаний, такие знания не хранятся в явной табличной форме и извлекаются только через генерацию ответа.

Reversal curse - эффект, при котором модель, обученная или хорошо отвечающая на факт в прямом направлении $A \rightarrow B$, не обязательно умеет восстановить обратный факт $B \rightarrow A$. Этот эффект показывает, что параметрическое знание LLM может быть асимметричным.

Multi-hop question answering - класс запросов вопросно-ответных систем, в которых для ответа требуется выполнить несколько связанных шагов: перейти от одной сущности к другой, объединить несколько фактов или проверить цепочку условий.

Многокритериальный запрос - пользовательский запрос, содержащий несколько ограничений одновременно: например, жанр, страну, год, тип объекта, рейтинг, автора, географическую принадлежность или отрицательное условие.

Околосинтетический критерий - намеренно добавленное условие, которое выглядит как обычный атрибут объекта, но с высокой вероятностью редко встречалось в текстах предобучения модели. Примером является номер визы или похожий регистрационный идентификатор в части cinema-запросов. Такие критерии нужны не для проверки энциклопедической памяти LLM, а для проверки способности поискового агента найти редкий факт во внешнем источнике и совместить его с остальными ограничениями запроса.

Поисковый агент - LLM-система, которая отвечает не только за счёт параметрической памяти модели, но и через обращение к ретриверу или поисковой системе. В проекте такие системы особенно важны для запросов с околосинтетическими и редкими критериями.

Реворд (reward) - численная оценка ответа модели, агрегирующая несколько сигналов качества: совпадение с эталонными ответами, соблюдение ограничений, отсутствие галлюцинаций, корректное поведение на запросах без корректного ответа и оценку LLM-as-a-Judge. В данной работе реворды используются как аналитические метрики для сравнения запусков и моделей.

List-style ответ - ответ, в котором модель должна выдать не один объект, а список объектов, удовлетворяющих условиям запроса. Для таких запросов особенно важны полнота, отсутствие галлюцинаций и соблюдение всех фильтров.

Wikidata - открытый многоязычный граф знаний, в котором сущности представлены идентификаторами QID, а отношения и свойства - предикатами. В работе Wikidata используется как основной источник истины для формального вычисления эталонных ответов.

Gold-ответ - эталонный ответ или множество ответов, полученное из внешнего проверяемого источника. В данном проекте эталонные ответы хранятся как QID и русские labels.

SPARQL - язык запросов к RDF-графам, используемый для формального извлечения сущностей из Wikidata по заданным ограничениям.

Запрос без корректного ответа - запрос, для которого корректное множество ответов должно быть пустым. Такие запросы нужны для проверки того, умеет ли модель отказываться от выдуманных ответов и объяснять несовместимость условий.

LLM-as-a-Judge - подход к структурированному разбору ответов, при котором сильная reasoning-модель извлекает из свободного ответа кандидатные сущности, сопоставляет их с эталонами и размечает выполнение критериев. В данной работе модель-судья не считает финальную оценку самостоятельно: итоговые метрики вычисляются отдельным модулем по фиксированным правилам.

ВВЕДЕНИЕ

За последние годы большие языковые модели стали основой многих прикладных систем обработки естественного языка. Они используются в поисковых ассистентах, диалоговых продуктах, инструментах поддержки пользователей, генерации рекомендаций и аналитических сценариях. От таких моделей ожидают не только грамматически связной генерации, но и устойчивого использования фактических знаний. Пользователь, задающий вопрос о фильмах, странах, книгах, научных объектах или исторических событиях, ожидает, что модель не просто сгенерирует правдоподобный текст, а выдаст проверяемый ответ, соответствующий всем ограничениям.

На практике фактические знания LLM оказываются неоднородными и асимметричными. Модель может знать один вариант формулировки факта, но ошибаться при обращении отношения, изменении направления вопроса или добавлении нескольких фильтров. В научной литературе это явление известно как *reversal curse*: обучение или знание факта вида «А является В» не гарантирует корректного ответа на запрос вида «какой А связан с В» [1]. Для продуктовых ассистентов такая ошибка особенно опасна, потому что пользователь часто формулирует запрос не в той форме, в которой факт встречался в обучающих данных.

Вторая важная проблема связана с многокритериальными и *multi-hop* запросами. В отличие от простого фактологического вопроса, такой запрос требует удержать несколько условий одновременно. Например, запрос «назови фильмы жанра криминальный фильм из США, вышедшие в 2015 году» формально является пересечением множества фильмов, множества криминальных фильмов, множества фильмов из США и множества фильмов с датой выхода в 2015 году. Если к этому добавить отрицания, условия по рейтингам, связи через актёров или *temporal constraints*, запрос становится ближе к запросу к базе данных, чем к обычному вопросу из QA-датасета.

Для русского языка подобные запросы покрыты слабо. Существуют полезные наборы данных, например RuBQ 2.0 для KBQA по Wikidata, многоязычный МКQA, а также англоязычные *complex QA* датасеты KQA Pro, Mintaka и 2WikiMultiHopQA. Однако прямое использование этих источников не решает запрос полностью. RuBQ и МКQA в основном содержат короткие одношаговые вопросы. KQA Pro богат логическими программами, но изначально англоязычен и часто с одним ответом. Mintaka и 2WikiMultiHopQA формально содержат *multi-hop* вопросы, но значительная часть примеров относится к поверхностным *bridge*-запросам или *comparison*-шаблонам. Поэтому возникает необходимость в отдельном русскоязычном бенчмарке, где сложность задаётся контролируемо, а эталонные ответы вычисляются формально.

Целью настоящей работы является разработка и исследование воспроизводимого русскоязычного бенчмарка для оценки способности LLM отвечать на сложные multi-hop и многокритериальные запросы, а также корректно вести себя в сценариях без корректного ответа. В качестве основного источника истины используется Wikidata, а внешние датасеты применяются не как готовая замена, а как доноры логических паттернов и вспомогательных срезов.

Объект исследования - большие языковые модели и LLM-системы, отвечающие на фактические пользовательские запросы. Предмет исследования - методы построения бенчмарка, позволяющего измерять качество ответов на многошаговые, многокритериальные, списочные и запросы без корректного ответа на русском языке.

Исследовательская гипотеза работы формулируется следующим образом: если построить бенчмарк вокруг формально проверяемого основного Wikidata-слоя и дополнить его внешним слоем сложных запросов и срезом запросов без корректного ответа для проверки галлюцинаций, то такой бенчмарк позволит обнаруживать систематические ошибки LLM - пропуск отдельных условий, слабость композиционного рассуждения, асимметрию фактического знания и галлюцинации при пустом множестве ответов - лучше, чем стандартные одношаговые QA-датасеты. Дополнительное эмпирическое ожидание состоит в том, что модели будут заметно устойчивее на простых контрольных запросах, чем на сложных многокритериальных запросах и запросах без корректного ответа.

Для достижения цели были выполнены следующие исследовательские и инженерные шаги: проанализированы существующие источники данных; разработана многоуровневая архитектура бенчмарка; собран и расширен основной Wikidata-слой; проведён отбор внешних сложных запросов; спроектирован срез запросов без корректного ответа; подготовлен контур инференса, обогащения gold-ответов, объединения срезов, структурированной LLM-as-a-Judge разметки, автоматического подсчёта метрик и ручной sanity-check валидации.

Практическая значимость работы состоит в том, что предлагаемый бенчмарк может использоваться как диагностический инструмент для команд, разрабатывающих русскоязычные LLM и ассистентов. Он позволяет не только сравнивать модели по итоговому баллу, но и раскладывать ошибки по типам: нарушение конкретного ограничения, неверный домен, выдуманная сущность, неполный список, некорректный отказ от ответа или отсутствие отказа в no-answer запросе.

ГЛАВА 1. ТЕКУЩЕЕ СОСТОЯНИЕ ОБЛАСТИ ПРОБЛЕМЫ

1.1. Проклятие обращения и фактические знания LLM

Большие языковые модели обучаются предсказывать следующий токен по предыдущему контексту. Такой способ обучения позволяет им усваивать огромное количество фактов, но не гарантирует, что эти факты будут доступны в любой логически эквивалентной форме. Если в обучающих данных часто встречается утверждение «А является В», модель может научиться продолжать контекст после А, но не обязательно сформирует симметричное или обратимое представление отношения.

Работа Berglund et al. описала этот эффект как reversal curse [1]. Авторы показали, что после обучения на вымышленных фактах вида «Uriah Hawthorne is the composer of Abyssal Melodies» модели хорошо отвечают на прямой вопрос о свойстве субъекта, но почти не переносят знание на обратный вопрос «Who composed Abyssal Melodies?». Важное наблюдение состоит в том, что проблема не сводится к отсутствию логической способности: если факт явно дан в контексте, модель может вывести обратное отношение. Проблема возникает именно при извлечении знания из параметров.

Для прикладных ассистентов это означает, что «модель знает факт» является слишком грубой характеристикой. Необходимо понимать, в каком направлении, при какой формулировке и в каком композиционном окружении знание доступно. Например, вопрос «кто режиссёр фильма X» и запрос «назови фильмы режиссёра Y, удовлетворяющие нескольким дополнительным условиям» могут иметь радикально разную сложность, хотя опираются на один и тот же набор фактов.

В данной работе reversal curse рассматривается не только в узком виде парных запросов $A \rightarrow B$ и $B \rightarrow A$, но и шире - как проявление асимметричности и хрупкости параметрического знания при обращении отношений, пересечении множеств и переходе от одного объекта к списку объектов. Это важно для реальных пользовательских запросов, где пользователь редко формулирует вопрос строго в «учебном» направлении. В этом смысле multi-hop и многокритериальные запросы можно рассматривать как более реалистичное пользовательское обобщение проблемы reversal curse. В реальном сценарии пользователь редко спрашивает только обратный факт в изолированной форме. Чаще он задаёт запрос вида «найди объекты, которые связаны с заданной сущностью и одновременно удовлетворяют нескольким дополнительным условиям». Например, вместо простого вопроса «кто режиссёр фильма X?» пользователь может спросить: «назови фильмы режиссёра Y в определённом жанре, вышедшие в заданный период и имеющие рейтинг выше заданного порога». Такой запрос требует не только обращения связи «фильм $\rightarrow$ режиссёр» в сторону «режиссёр $\rightarrow$ фильмы», но и пересечения результата с другими множествами объектов.

Поэтому в данной работе multi-hop и multi-constraint запросы используются как практическая форма проверки того, насколько модель умеет восстанавливать сущности по обратным и составным условиям, а не только воспроизводить отдельные факты в привычном направлении.

1.2. Multi-hop и многокритериальные запросы как класс запросов

Multi-hop question answering обычно предполагает, что для ответа нужно выполнить несколько связанных шагов рассуждения: перейти от одной сущности к другой, восстановить промежуточную связь, а затем проверить итоговое свойство. Например, чтобы ответить на вопрос «где родился режиссёр фильма, получившего такую-то премию», нужно сначала найти фильм или режиссёра, затем перейти к месту рождения. В многокритериальных запросах сложность может проявляться иначе: модель должна не столько построить длинную цепочку, сколько корректно пересечь несколько множеств объектов.

Для таких запросов ошибка модели часто бывает частичной. Один объект из списка может удовлетворять жанру и году, но не стране; другой может быть объектом правильного типа, но нарушать дополнительное отношение; третий может быть тематически похожим на правильные ответы, но не соответствовать ни одному эталонному ответу. Поэтому простая проверка по точному совпадению одной строки ответа плохо подходит для этой постановки: корректных ответов может быть несколько, модель может вернуть только подмножество правильных ответов, а для анализа качества важно понимать, какие именно ограничения были нарушены.

Именно поэтому в проекте используется связка естественно-языкового запроса и формального слоя. Модель видит только русский текст запроса, а исследовательский контур хранит constraints, SPARQL-запрос, QID эталонных ответов, лейблы и при необходимости ASK-валидатор для проверки конкретной сущности.

1.3. Существующие датасеты и ограничения текущей практики

Существующие бенчмарки покрывают разные стороны QA и reasoning, но не полностью совпадают с целью данной работы. Wikidata является удобным источником истины благодаря многоязычным лейблам, явным QID и возможности выполнять SPARQL-запросы [2]. Однако сама по себе Wikidata не является готовым пользовательским бенчмарком: нужно спроектировать шаблоны, контролировать сложность, фильтровать плохие лейблы и отделять реальные пустые множества от технических ошибок генерации.

RuBQ 2.0 - важный русскоязычный ресурс для KBQA по Wikidata. Он содержит вопросы, ответы и SPARQL-запросы [3]. Однако анализ в рамках проекта показал, что большинство таких вопросов относится к одношаговым фактологическим сценариям: «где родился человек», «кто автор произведения», «когда произошло событие», «в какой

стране находится объект». Они полезны как доноры русских формулировок и relation-паттернов, но недостаточно сложны для основного слоя бенчмарка.

МКQA предлагает многоязычную open-domain QA постановку: 10 тысяч вопросно-ответных пар, выровненных по 26 языкам, то есть 260 тысяч языковых вариантов [4]. Его русская часть полезна для анализа естественных формулировок, но датасет не предоставляет такой же строгой структуры SPARQL и не ориентирован специально на многокритериальные запросы со списочным ответом.

KQA Pro значительно ближе к целевой постановке по сложности. Он содержит около 120 тысяч complex KBQA вопросов, для которых предоставлены KoPL-программы и SPARQL-запросы [5]. В рамках проекта именно KQA Pro оказался лучшим внешним источником логических паттернов: multi-hop chains, qualifiers, temporal constraints, comparison, superlative и set operations. Однако он англоязычен, часто синтетичен по стилю и в отобранном подмножестве в основном с одним ответом, поэтому не может напрямую заменить основной Wikidata-слой.

Mintaka позиционируется как complex, natural и multilingual QA датасет: 20 тысяч английских вопросов, аннотированных сущностями Wikidata и переведённых на несколько языков [6]. Он включает типы complexity вроде superlative, intersection и multi-hop. Тем не менее практический анализ показал, что наличие метки multihop само по себе не гарантирует высокой композиционной сложности: многие вопросы являются поверхностными bridge-запросами.

2WikiMultiHopQA строился как датасет для комплексной оценки шагов рассуждения, использующий структурированные и неструктурированные данные и явные evidence paths [7]. Он полезен для comparison и bridge-comparison reasoning, но в рамках данного проекта оказался слишком однотипным и в основном одноответным для прямого расширения основного слоя со списочными ответами.

Таким образом, анализ существующих источников приводит к выводу: ни один внешний датасет не закрывает целевую постановку напрямую. Основным слоем должен оставаться контролируемый Wikidata-слой, а внешние источники целесообразно использовать как вспомогательные доноры паттернов и диагностические срезы.

1.4. Постановка исследовательской проблемы и гипотезы

Проблема исследования состоит в отсутствии русскоязычного воспроизводимого бенчмарка, который одновременно проверяет multi-hop reasoning, многокритериальные ответы списком, обращённые фактические связи и корректное поведение модели при отсутствии ответа. Стандартные одношаговые QA-датасеты часто не выявляют такие ошибки, потому что модель может успешно отвечать на простые вопросы, но проваливаться при пересечении нескольких условий. Иными словами, целевая постановка работы расширяет классический reversal curse от проверки одной обращённой пары фактов к проверке реального пользовательского сценария, в котором

модель должна восстановить сущность или список сущностей по набору обратных, составных и фильтрующих условий.

Основная гипотеза была сформулирована как методологически-эмпирическая. Методологическая часть утверждает, что бенчмарк должен иметь формальный слой источника истины: без constraints, QID и SPARQL невозможно надёжно отличить ошибку модели от неоднозначности вопроса. Эмпирическая часть утверждает, что увеличение числа и разнородности условий будет приводить к росту доли ошибок: модель будет чаще игнорировать отдельные constraints, подменять сущности похожими, выдавать устаревшие или несуществующие объекты и галлюцинировать в запросах без корректного ответа.

Проверка гипотезы в рамках ВКР понимается не как дообучение модели, а как построение и валидация измерительного инструмента. Если бенчмарк позволяет разложить запросы по сложности и типам ошибок, а также показывает, почему внешние датасеты должны использоваться как вспомогательные слои, то он достигает своей исследовательской цели.

1.5. Выводы по главе 1

В первой главе показано, что reversal curse и многокритериальные запросы являются важными источниками ошибок LLM. Для русского языка отсутствует готовый бенчмарк, который объединяет эти аспекты в формально проверяемой постановке. Существующие источники данных полезны, но каждый имеет ограничения: одни слишком просты, другие англоязычны, третьи однотипны или с одним ответом. Это обосновывает необходимость разработки многоуровневого бенчмарка, описанного в следующей главе.

ГЛАВА 2. ОПИСАНИЕ РЕШЕНИЯ

2.1. Требования к бенчмарку и общий метод исследования

Разрабатываемый бенчмарк должен удовлетворять нескольким требованиям. Во-первых, он должен иметь внешний источник истины, независимый от тестируемой модели. Во-вторых, каждый запрос должен быть воспроизводимым: исследователь должен иметь возможность восстановить условия, эталонные ответы и способ проверки. В-третьих, бенчмарк должен быть русскоязычным на уровне пользовательских формулировок. В-четвёртых, он должен различать разные типы сложности, а не смешивать простые фактологические вопросы, сложные запросы со списочным ответом и провокации без корректного ответа в один общий балл.

Основной метод исследования состоит в генерации и анализе запросов на пересечении естественного языка и формального запроса. Для каждого примера строится текстовый запрос на русском языке, но его смысл хранится в виде constraints. Эти constraints затем используются для SPARQL-запроса к Wikidata и вычисления эталонных ответов. Такой подход позволяет избежать ручной разметки каждого ответа и делает оценку масштабируемой.

Внешние датасеты используются по другой логике. Они не принимаются целиком как основной слой бенчмарка, а проходят анализ, фильтрацию и ручной просмотр. Для каждого источника оценивается, что именно он может дать проекту: готовые русские формулировки, сложные логические паттерны, примеры с одним ответом, запросы с акцентом на сравнение или идеи для будущих шаблонов.

2.2. Многоуровневая архитектура датасета

В результате работы была выбрана многоуровневая архитектура. Главная причина такого решения - неоднородность запросов. Нельзя честно сравнивать качество модели на лёгких one-hop вопросах, сложных многокритериальных запросах со списочным ответом, внешних запросах с одним ответом и no-answer провокациях как на одной однородной массе. Каждый слой должен иметь собственное назначение и отдельную аналитику.

Архитектура включает четыре ключевых уровня. Первый - основной Wikidata-слой, главный слой источника истины и основной источник метрик. Второй - простой контрольный слой более простых запросов. Третий - внешний слой сложных запросов, куда входят отобранные сложные запросы с одним ответом и паттерны из KQA Pro и Mintaka; 2WikiMultiHopQA был дополнительно проанализирован, но не включён в финальный слой из-за однотипности и недостаточной диагностической ценности для целевой постановки. Четвёртый - срез запросов без корректного ответа для проверки галлюцинаций, где корректных ответов быть не должно.

Слой	Ориентировочный объём	Роль в бенчмарке	Ключевые особенности
Wikidata core_hard	около 800	основной слой	русскоязычные multi-hop и multi-constraint запросы, ответы списком, формальные эталонные ответы через Wikidata
Wikidata easy_shallow	около 200	простой контрольный срез	проверка базовой способности модели отвечать на простые случаи и оценка разрыва easy-hard
External kqapro complex_single	75	вспомогательный срез	сложные запросы с одним ответом: temporal, qualifier, logical_set, comparison
External mintaka selected	15	точечный вспомогательный срез	отдельные естественные multi-hop примеры после ручного отбора
zero_gold hallucination_probe	около 100	срез без корректного ответа	near-miss запросы с пустым множеством корректных ответов

2.3. Wikidata-core: генерация, ограничения, gold-ответы и уровни сложности

Wikidata-слой является основой проекта. Его преимущество в том, что запросы можно строить контролируемо: выбирается домен, набор допустимых свойств, уровень сложности и ограничения. Затем по этим ограничениям формируется SPARQL-запрос, который возвращает множество сущностей-кандидатов. Каждая сущность хранится как QID и русское название, что позволяет одновременно проводить строгую проверку и отображать ответы человеку.

На текущем этапе основной Wikidata-слой содержит около 1000 русскоязычных запросов с эталонными ответами. Они покрывают примерно 20-25 доменов. Среди доменов рассматриваются фильмы и сериалы, книги, страны, географические объекты, аэропорты, университеты, блюда, смартфоны, автомобили, программное обеспечение, картины, музыкальные альбомы, видеоигры, музеи, люди и другие области. Набор условий включает жанр, страну, год или период, рейтинг, автора, участника, географическую привязку, тип объекта, принадлежность к категории, отрицания и дополнительные связи.

Генератор работает не как набор ручных примеров, а как воспроизводимый пайплайн. Для каждого домена определяются релевантные свойства Wikidata и пулы допустимых значений. Затем сэмплируются combinations of constraints, строится SELECT DISTINCT запрос и вычисляются эталонные ответы. Для части запросов дополнительно сохраняется ASK-валидатор. Он нужен в тех случаях, когда полный список эталонных ответов ограничен LIMIT: даже если сущность не попала в сохранённый gold-list, её можно проверить отдельно по тем же формальным условиям.

Отдельная практическая проблема — качество русских названий. Для оценки русскоязычной модели недостаточно иметь только QID: ответ должен быть сопоставим с тем, как пользователь или модель реально назовёт объект по-русски. Поэтому для доменов вне кино дополнительно используются label, alias, ruwiki sitelink и монолингвальные названия. Это уменьшает число случаев, когда в эталонах остаются английские, странно транслитерированные или просто нечитаемые варианты.

После первичной генерации в пайплайн был добавлен отдельный этап обогащения уже существующих Wikidata gold-ответов. Этот этап не меняет смысл запроса и не заменяет эталоны вручную. Его задача более техническая: собрать вокруг каждой gold-сущности больше вариантов названия и признаков, которые помогут при проверке ответа модели. Для сущности сохраняются русские лейблы, возможные альтернативные написания, английские варианты, элиасы и названия из связанных страниц. На практике это важно, потому что один и тот же фильм, книга или географический объект может быть назван моделью в нескольких формах: официальным русским названием, оригинальным названием, транслитерацией или распространённым элиасом.

Обогащение gold-ответов также помогает отличать две разные ситуации: модель назвала сущность, которая действительно совпадает с эталонной, но записана иначе, и модель назвала похожую, но неверную сущность. Без такого слоя автоматическая оценка могла бы занижать качество из-за вариантов написания или, наоборот, ошибочно засчитывать тематически похожие ответы. Поэтому enrichment-этап повышает устойчивость оценки и делает последующую работу модели-судьи более надёжной: судья получает не только один gold label, но и расширенный контекст для нормализации ответа.

При этом обогащение не отменяет формальный слой проверки. Основным источником истины остаются constraints, QID и SPARQL/ASK-валидаторы. Если модель выдала объект, которого нет среди gold labels, но который потенциально может удовлетворять условиям из-за неполноты Wikidata или усечения списка, такой объект не засчитывается автоматически, а попадает в отдельную диагностическую категорию possibly_correct_not_in_gold для ручного анализа или дополнительной проверки.

Уровни сложности L1-L5 используются для системного варьирования трудности. В простых уровнях запрос обычно содержит меньше условий и имеет достаточно много эталонных ответов. В более сложных уровнях добавляются редкие комбинации, дополнительные отношения, отрицания, рейтинги, temporal constraints и multi-hop

переходы. В некоторых доменах конкретная семантика уровней отличается, но общая идея остаётся одинаковой: от частотных и прямых условий к редким, композиционным и хрупким.

Поле записи	Назначение
Id	уникальный идентификатор примера, например cinema_11_0000
domain	домен запросы: cinema, books, geo, countries и т. д.
complexity	уровень сложности L1-L5
query_text_ru	русский текст запроса, который видит модель
constraints	структурированное описание условий
requested_count	сколько объектов просит пользователь
gold_answer_qids	список QID эталонных ответов
gold_answer_labels_ru	русские названия эталонных ответов
sparql_query	исходный SELECT-запрос или его служебный эквивалент
gold_truncated	флаг, что gold был ограничен LIMIT и может быть неполным
ask_validator_sparql	ASK-шаблон для проверки отдельной кандидатной сущности
template_family template_id	служебные признаки типа шаблона

2.4. Околосинтетические критерии и проверка поискового поведения

В бенчмарке также выделен отдельный тип условий — околосинтетические критерии. Это не произвольные искусственные загадки, а формально проверяемые признаки, связанные с реальными или близкими к реальным атрибутами объекта. При этом они выбираются так, чтобы с высокой вероятностью не быть хорошо представлены в параметрической памяти LLM. Например, в cinema-запросах таким критерием может быть номер визы, прокатного удостоверения, редкий регистрационный идентификатор или другой малораспространённый атрибут, который редко встречается в обычных текстах и пользовательских вопросах.

Такие условия нужны именно как стресс-тест. Если запрос состоит только из популярных признаков — например, жанра, страны и года выхода фильма, — сильная модель иногда может ответить по памяти или по устойчивым ассоциациям. Это полезно для проверки фактического знания, но плохо отделяет обычную параметрическую память от полноценного поискового поведения. Редкий критерий меняет постановку: ответ уже трудно просто угадать, и модели или агенту приходится обращаться к внешнему источнику.

Важно, что околосинтетический критерий не должен превращать запрос в произвольную загадку. Он должен быть проверяемым, однозначным и хранимым в

метаданных запроса: например, как отдельное поле `constraints` или как флаг `retrieval_required / semisynthetic_criterion`. В идеале для такого критерия дополнительно хранится источник проверки или способ валидации. Тогда при анализе можно разделять ошибки разных типов: модель могла правильно найти фильмы нужного жанра и периода, но не проверить номер визы; могла найти редкий идентификатор, но нарушить страну; или могла вообще проигнорировать околосинтетическое условие.

Такой слой особенно важен для сравнения чистых LLM и поисковых агентов. YandexGPT Pro 5.1 без поиска и similar baseline-модели проверяются в режиме параметрической памяти и reasoning. Алиса на Поиске и Нейроалиса с поиском, напротив, являются агентными системами с ретривером, поэтому для них ожидается преимущество именно на запросах с редкими и околосинтетическими условиями. Если поисковый агент не улучшает качество на этом срезе, это указывает на проблему не столько языковой модели, сколько поискового маршрута, извлечения доказательств или объединения найденного факта с остальными критериями.

В итоговой аналитике такие запросы рекомендуется считать отдельным срезом, а не смешивать без маркировки с обычным основным Wikidata-слоем. Это позволяет честно отвечать на два разных вопроса: насколько модель знает популярные факты из памяти и насколько агентная система умеет пользоваться поиском для редких фактов. Для ВКР этот слой является важным расширением постановки: бенчмарк становится не только тестом на reversal curse и multi-hop reasoning, но и инструментом диагностики поискового поведения и объединения фактов.

2.5. Анализ внешних источников и формирование слоя сложных запросов

Внешние источники анализировались не как готовая замена основному Wikidata-слою, а как способ расширить бенчмарк за счёт дополнительных логических паттернов. Главный вопрос состоял не в том, содержит ли датасет формальную метку multi-hop, а в том, помогает ли он получить запросы, близкие к целевой постановке: русскоязычные или переводимые на русский язык, проверяемые по эталонным ответам, достаточно сложные, не сводящиеся к одному очевидному факту и пригодные для отдельного вспомогательного среза. Поэтому каждый источник проходил одинаковую исследовательскую схему: импорт, нормализацию, первичную эвристическую оценку, фильтрацию, группировку по типам рассуждения, ручной sanity-check и решение о роли источника в итоговой архитектуре.

В результате была принята многоуровневая стратегия. Основным источником истины остаётся основной Wikidata-слой, поскольку он даёт формальные SPARQL-запросы, QID эталонных ответов, русские лейблы и возможность контролировать сложность. Внешние датасеты используются осторожнее: они поставляют сложные логические схемы, редкие формулировки и отдельные диагностические срезы, но не смешиваются с основным слоем без маркировки. Это принципиально важно для интерпретации

метрик: нельзя одинаково трактовать ошибку на Wikidata-запросе со списочным ответом и ошибку на одноответном запросе из внешнего англоязычного корпуса.

2.5.1. Общая методика анализа внешних датасетов

Для каждого внешнего источника был создан отдельный ноутбук или цепочка ноутбуков. Такое разделение сделано намеренно: разные корпуса имеют разные схемы, разные типы эталонных ответов и разные проблемы качества. Единый монолитный импорт быстро привёл бы к потере информации о происхождении запроса, поэтому каждый источник сначала обрабатывался автономно, а уже затем его кандидаты приводились к общей схеме проекта.

Типичный ноутбук импорта выполняет несколько операций. Сначала он загружает исходный корпус из доступного источника: через библиотеку `datasets`, raw JSON/JSONL, CSV или Parquet. Далее поля приводятся к единому виду: `source`, `source_id` или `benchmark_id`, исходный текст запроса, язык, ответ, число ответов, тип запроса, доступные программы рассуждения, SPARQL или другие формальные представления. После этого ноутбук создаёт технические признаки: длину запроса, количество ответов, наличие структурированной программы, наличие Wikidata/QID-связей, тип рассуждения и признаки потенциальной пригодности для восстановления.

На следующем шаге применялась фильтрация. Убирались или понижались в приоритете запросы, которые плохо соответствуют целевой постановке: `yes/no verification`, URL/email/postal-code запросы, слишком простые фактологические вопросы, `count-only` формулировки без содержательного списка объектов, поверхностные `bridge`-запросы с одним очевидным переходом, а также сильно шаблонные или повторяющиеся `surface` паттернов. Для сохранения разнообразия кандидаты группировались по `program signature`, `question type` или `surface template`, чтобы таблица для ручной проверки не состояла из десятков почти одинаковых примеров.

Финальный шаг - ручной `sanity-check`. После автоматической фильтрации я просматривал короткие списки кандидатов и удалял невалидные, слишком простые, плохо переводимые или неинтересные запросы. Важная часть этой проверки заключалась в том, что формальная метка `multi-hop` сама по себе не считалась достаточным основанием для включения. Если запрос требовал два шага технически, но по смыслу был простым одноатрибутным вопросом, он не попадал в основной внешний слой сложных запросов.

2.5.2. RuBQ и MKQA: русские формулировки и ограничения для `core`-слоя

Первыми внешними источниками были рассмотрены RuBQ 2.0 и MKQA. Для них был подготовлен ноутбук `02_import_rubq_mkqa_ru_nodedup.ipynb`. Его назначение состояло в том, чтобы проверить, можно ли использовать существующие русскоязычные или

мультязычные QA-корпуса как источник дополнительных multi-hop запросов, а также извлечь из них полезные русские формулировки и relation-паттернов.

Ноутбук загружает RuBQ 2.0 и МКQA, извлекает русский слой, нормализует поля и приводит записи к единой промежуточной схеме. Для RuBQ важными полями являются текст вопроса, ответ, Wikidata-связи и SPARQL-представление, если оно доступно. Для МКQA акцент делается на русском варианте вопроса и ответа, поскольку строгая SPARQL-структура там выражена слабее. После нормализации ноутбук выполняет дедупликацию, чтобы одинаковые или почти одинаковые вопросы не попадали в проверку несколько раз.

Отдельно в ноутбуке вводится разметка ролей кандидатов. Записи могли получать роль `direct_ru_candidate`, если они потенциально пригодны как готовые русские QA-запросы; `reconstruction_candidate`, если их можно использовать для восстановления или генерации похожих structured-запросов; `wording_donor`, если сам запрос слишком простой, но его русская формулировка полезна как пример естественного языка. Такой подход позволил не выбрасывать корпус полностью, даже если он не подходит для сложного основного слоя напрямую.

Результаты анализа показали, что RuBQ полезен технически: в нём есть русские вопросы, ответы, связь с Wikidata и в части случаев SPARQL. Однако содержательно большинство вопросов оказалось одношаговыми фактологическими запросами: где родился человек, кто автор произведения, когда произошло событие, в какой стране находится объект, кто основал организацию. Такие запросы хороши для KBQA, но слабо проверяют удержание нескольких ограничений и композиционное reasoning.

МКQA оказался полезен как источник multilingual phrasing и русских переформулировок, но не как источник готовых multi-hop запросов. Многие вопросы также относятся к open-domain QA с одним коротким ответом. Кроме того, отсутствие такого же строгого SPARQL-слоя, как в основном Wikidata-слое, усложняет автоматическую проверку. Поэтому RuBQ и МКQA не были включены как основной внешний слой сложных запросов; их роль ограничена donor-функцией: русские паттерны формулировок, простые relation-паттерны и материал для сравнения с более сложными источниками.

2.5.3. KQA Pro: основной внешний донор сложных reasoning-паттернов

Наиболее важным внешним источником оказался KQA Pro. В отличие от RuBQ/МКQA, этот корпус содержит не только текстовые вопросы и ответы, но и явные структурные представления reasoning: KoPL-программы, SPARQL, логические операции, qualifiers, temporal constraints, comparison, superlative и multi-hop chains. Поэтому KQA Pro рассматривался как главный кандидат на формирование внешнего слоя сложных запросов.

Для работы с KQA Pro была создана цепочка из трёх ноутбуков. Первый - 03_import_kqapro_fixed.ipynb - отвечает за импорт и нормализацию. Обычная загрузка через datasets оказалась нестабильной из-за особенностей dataset scripts, поэтому в ноутбук был добавлен резервный режим на прямую загрузку raw JSON. Это важная инженерная часть: импорт должен быть воспроизводимым и не зависеть от того, отработает ли внешний загрузчик. В результате было загружено 106 173 записи: 94 376 train и 11 797 val.

На этапе импорта ноутбук извлекает исходный вопрос, ответ, split, program, SPARQL, типы операторов KoPL и служебные идентификаторы. Дополнительно вычисляются признаки сложности: длина программы, наличие нескольких hop-ов, наличие comparison или superlative, наличие qualifiers, temporal constraints, logical set operations и count. По этим признакам было выделено около 72 792 сложных кандидатов. Это не означает, что все они подходят для финального слоя, но показывает, что KQA Pro содержит большой запас сложных структур.

Второй ноутбук - 04_filter_kqapro_for_reconstruction.ipynb - выполняет содержательную фильтрацию. Он убирает verification / yes-но запросы, поскольку они плохо соответствуют формату ответа списком или сущностью; исключает website, URL, email, postal code и похожие технические ответы; понижает приоритет count-only запросов, где модель должна назвать только число; считает оценку пригодности запроса для восстановления, то есть оценивает, насколько запрос можно перевести и использовать в целевом бенчмарке без потери смысла. Также ноутбук строит program signature, группирует похожие программы и формирует банк паттернов.

Оценка пригодности запроса для восстановления в этом контексте является не модельной метрикой, а эвристической оценкой пригодности запроса для дальнейшей реконструкции. Выше оцениваются примеры, где есть явная композиционная структура, понятный ответ, содержательные сущности и условия, которые можно перевести на русский. Ниже оцениваются примеры со слишком техническим ответом, неудачной формулировкой, избыточно синтетическим стилем или слабой диагностической ценностью. После этой оценки ноутбук формирует таблицу для ручной проверки, в которой кандидаты сбалансированы по типам рассуждения: multi-hop, logical_set, qualifier, comparison_or_superlative, temporal и count.

После автоматической фильтрации был получен короткий список кандидатов из 180 записей. Затем проводился ручной просмотр: я отсматривал исходный вопрос, ответ, программу, тип reasoning и потенциальную пригодность для русского benchmark-a. На этом этапе удалялись вопросы, которые формально сложные, но плохо звучат после перевода, имеют слишком специфичный технический ответ, дублируют уже выбранные паттернов или не дают дополнительной диагностической ценности относительно основного Wikidata-слоя. По итогам ручного отбора было оставлено 62 сильных запроса.

Третий ноутбук - 05_prepare_kqapro_selected_for_translation_pipeline_with_json.ipynb - отвечает за упаковку выбранных KQA Pro-запросов. Он берёт вручную отобранные `benchmark_id`, подтягивает полные исходные записи, проверяет отсутствие потерь и дублей, а затем формирует файлы `kqapro_selected_for_translation_pipeline.json` и `kqapro_translation_minimal.json`, а также CSV/JSONL версии для просмотра и интеграции. В полной версии сохраняются исходные `question/answer`, `program`, SPARQL, `reasoning family` и служебная метадата; в `minimal`-версии остаётся только то, что нужно для перевода и дальнейшего инференса.

Содержательный вывод по KQA Pro следующий: это лучший внешний источник для проекта, но не замена основному Wikidata-слою. Все 62 отобранные записи в основном являются сложными запросами с одним ответом. Они хорошо проверяют композиционное `reasoning`, `temporal constraints`, `qualifiers` и `comparison`, но не продолжают напрямую постановку основного Wikidata-слоя со списочным ответом, где корректных ответов может быть несколько. Поэтому KQA Pro включается как отдельный вспомогательный срез `external_kqapro_complex_single` и как банк логических паттернов для будущего расширения генератора.

2.5.4. Mintaka: формальная multi-hop метка и проблема shallow bridge

Следующим источником был рассмотрен Mintaka. Он интересен тем, что содержит `complex questions` и отдельные `multi-hop` метки. Для него были подготовлены ноутбуки `03c_import_mintaka_multihop.ipynb` и `04c_filter_mintaka_for_reconstruction.ipynb`. Цель анализа состояла в том, чтобы проверить, можно ли из Mintaka получить компактный, но качественный вспомогательный слой с более естественными вопросами.

Ноутбук `03c_import_mintaka_multihop.ipynb` выполняет первичный импорт Mintaka, извлекает вопросы, ответы, типы и признаки, связанные с `multi-hop`. После нормализации было получено около 2000 вопросов, помеченных как `multihop`. На этом этапе корпус выглядел перспективно: в нём есть реальные вопросные формулировки, разные домены и часть запросов действительно требует перехода между сущностями.

Однако дальнейший ручной просмотр показал важную проблему: метка `multihop` не гарантирует содержательной сложности. Многие вопросы относятся к поверхностным `bridge`-запросам. Например, запрос может требовать сначала найти исполнителя песни, а затем назвать место его рождения; или перейти от фильма к продюсеру; или от команды к игроку. Формально здесь есть два шага, но по смыслу это один простой мост и извлечение одного атрибута. Такой запрос слабее проверяет одновременное удержание нескольких ограничений, чем целевые `multi-constraint` запросы `benchmark-a`. Ноутбук `04c_filter_mintaka_for_reconstruction.ipynb` был написан именно для отделения более перспективных Mintaka-кандидатов от массы поверхностных `bridge`-запросов. Он понижает поверхностные `bridge questions`, повышает приоритет `list-like` ответов, учитывает `num_answers >= 2`, пытается выделять `temporal`, `comparison` и `intersection`

паттернов, а также группирует вопросы по surface template. Это нужно, чтобы в ручную проверку не попадали десятки однотипных формулировок вроде “Where was the singer of X born?”.

Даже после фильтрации таблица для ручной проверки осталась достаточно шумной. Тем не менее в Mintaka были найдены точно полезные примеры: вопросы про первых сенаторов нового штата США, жён президентов США, которые были матерями президентов, победителей первых спортивных турниров, screenplay / adaptation chains и некоторые nested superlative / temporal паттернов. Итоговая роль Mintaka - небольшой вспомогательный слой из примерно 5-15 запросов, если они пройдут ручную проверку и перевод. В основной слой бенчмарка Mintaka не включается.

2.5.5. 2WikiMultiHopQA: большой объём и узкая диагностическая польза

Последним крупным внешним источником был рассмотрен 2WikiMultiHopQA. Для него были подготовлены ноутбуки 03e_import_2wikimultihop_fixed.ipynb и 04e_filter_2wikimultihop_for_reconstruction_fixed_v5.ipynb. Изначально ожидалось, что большой multi-hop корпус даст много сложных кандидатов, однако анализ показал более узкую применимость.

Ноутбук 03e_import_2wikimultihop_fixed.ipynb отвечает за корректную загрузку датасета и нормализацию структуры. Первоначальная версия ожидала JSON/JSONL, но фактически данные были доступны в Parquet. После исправления загрузки были импортированы 192 606 записей: 167 454 train, 12 576 dev и 12 576 test. Для каждой записи сохранялись вопрос, ответ, тип, supporting facts или доступная структурная информация, а также признаки question type.

В корпусе встречаются типы compositional, comparison, bridge_comparison и inference. На первый взгляд это хорошо соответствует multi-hop постановке. Но первичный анализ показал, что значительная часть запросов сосредоточена вокруг однотипных comparison- и bridge-comparison-шаблонов: кто старше, кто умер раньше, какой фильм вышел раньше, какой режиссёр связан с каким фильмом. Это полезно для диагностики с акцентом на сравнение reasoning, но не даёт широкого разнообразия многокритериальных запросов со списочным ответом.

Ноутбук 04e_filter_2wikimultihop_for_reconstruction_fixed_v5.ipynb выполняет более жёсткую фильтрацию. Он понижает bridge_comparison, удаляет или штрафует однотипные movie/director older/earlier/died first паттерны, понижает поверхностные bridge-запросы, понижает короткие одноответные цепочки и пытается сохранить более разнообразные логические паттерны. В ходе экспериментов слишком жёсткий фильтр сначала отправил все 1579 сильных кандидатов в drop, поэтому был добавлен резервный режим, позволяющий собрать более мягкий набор кандидатов для ручного просмотра.

После исправлений было получено 399 отфильтрованных кандидатов, банк паттернов из 366 паттернов и короткий список на 35 записей для ручной проверки. Финальный

короткий список состоял из 25 comparison и 10 bridge_comparison запросов; все они были с одним ответом. Поэтому 2WikiMultiHopQA полезен как диагностический срез для рассуждения при сравнении и bridge reasoning, требующего поиска, но не подходит как основной внешний источник для расширения основного слоя бенчмарка. Его можно использовать точечно, чтобы посмотреть, насколько модели справляются со сравнением и мостовыми цепочками, но смешивать эти запросы с основным Wikidata-слоем без отдельной метки нельзя.

2.5.6. Итоговая роль внешнего слоя сложных запросов

По результатам анализа внешние источники были разделены по ролям. KQA Pro стал главным донором внешнего слоя сложных запросов: из него отобрано 62 сложных запроса с одним ответом и сохранён банк композиционных паттернов. Mintaka признан ограниченно полезным источником: он даёт небольшой диагностический срез, но требует ручного отбора и не обеспечивает масштабного расширения основного слоя. 2WikiMultiHopQA был отдельно проанализирован, однако не включён в финальный датасет из-за однотипности, преобладания простых comparison / bridge-comparison шаблонов и слабого соответствия целевой list-style multi-constraint постановке. RuBQ и МКQA используются преимущественно как источники формулировок и простых KBQA relation-паттернов.

Главный методологический вывод состоит в том, что внешние датасеты нельзя использовать только по названию или по заявленной метке multi-hop. Для целевого бенчмарка важна не формальная многошаговость, а диагностическая ценность запроса: сколько условий нужно удерживать, есть ли проверяемый эталон, насколько запрос переводим на русский, не является ли он поверхностным bridge и какую ошибку модели он помогает обнаружить. Поэтому внешний слой сложных запросов строится как отобранный и размеченный слой, а не как механическое объединение всех доступных multi-hop корпусов.

Такой подход делает итоговую архитектуру более честной. Основной Wikidata-слой остаётся главным слоем источника истины и главным источником многокритериальных запросов со списочным ответом. Внешний слой сложных запросов добавляет сложные запросы с одним ответом и паттерны сравнения, уточняющих условий и временных ограничений. Срез запросов без корректного ответа проверяет отказ от галлюцинаций. В совокупности эти слои позволяют анализировать не одну усреднённую метрику, а разные типы поведения LLM и поисковых агентов.

Источник	Что было сделано	Итоговая роль
Wikidata	генерация русскоязычных запросов, constraints, SPARQL, QID, labels, ASK validators	основной truth layer и benchmark-core
RuBQ / МКQA	импорт, дедупликация, выделение русского слоя, нормализация	wording donor и источник простых KBQA patterns
KQA Pro	импорт 106 173 записей, выделение 72 792 complex candidates, shortlist 180, ручной отбор 62	главный внешний донор complex reasoning patterns
Mintaka	анализ около 2000 multihop-вопросов, фильтрация shallow bridge	малый auxiliary layer после ручного отбора
2WikiMultiHopQA	импорт 192 606 записей, фильтрация до 399 candidates и 35 shortlist	diagnostic split для comparison-heavy reasoning

2.6. Срез запросов провокаций на галлюцинации

Отдельной частью решения является срез запросов без корректного ответа. В обычных QA-запросах предполагается, что корректный ответ существует. Однако реальные пользователи могут задавать вопросы с несовместимыми условиями, устаревшими предпосылками или слишком узкими пересечениями. Если модель в таких случаях всегда пытается выдать список, она будет галлюцинировать. Поэтому в бенчмарке нужен специальный слой, где правильное поведение - явно сообщить, что подходящих объектов нет, и по возможности объяснить причину.

Первоначальная автоматическая логика складывала в отдельный zero-набор все записи с `gold_answer_qids == []`. Анализ показал, что это неверно: в такой набор попадали failed/skipped записи, технические сбои генератора, шаблоны с constraints `{"failed": true}`, пустые заглушки и реальные empty-gold. Поэтому было принято решение различать пустое множество ответов как техническую ошибку и пустое множество ответов как содержательный запрос без корректного ответа.

Для запросов без корректного ответа были выделены типы невозможности: `hard_contradiction`, `temporal_impossibility`, `historical_impossibility`, `mutually_exclusive_filters`, `empty_intersection`, `category_contradiction`, `future_impossibility` и `fake_entity_or_impossible_attribute`. Для каждого такого запроса нужно хранить `answerability = no_answer`, `no_answer_type`, `no_answer_reason` и `expected_behavior`.

Первая ручная генерация кандидатов для среза запросов без корректного ответа оказалась слишком очевидной: аэропорты на Марсе, страны на Луне, смартфоны до

изобретения электричества. Такие примеры плохо проверяют LLM, потому что отказ очевиден. Поэтому стратегия была изменена на near-miss negative examples: запросы выглядят правдоподобно, но содержат скрытое противоречие в датах, категориях или отношениях.

Примеры хороших паттернов запросов без корректного ответа: фильм основан на романе, опубликованном позже выхода фильма; сериал является original Netflix, но вышел до эпохи Netflix; режиссёр родился после выхода фильма; фильм получил премию, которой ещё не существовало; страна имеет морскую границу при отсутствии выхода к морю; озеро одновременно относится к несовместимым бассейнам. В рамках работы было подготовлено около 92 кандидатов, которые должны пройти валидацию через Wikidata-пайплайн. Целевой размер слоя - около 100 запросов.

2.7. Формат записи, репозиторий и воспроизводимость

Репозиторий проекта организован по источникам данных и этапам обработки. Директория wikidata содержит основной пайплайн генерации запросов и эталонных ответов через Wikidata, основной ноутбук генерации, выходные JSON/JSONL-артефакты и отдельный ноутбук для обогащения уже существующих gold-ответов. Директория kqargo содержит ноутбуки и артефакты для импорта, фильтрации и подготовки KQA Pro. Директория gubq_mkqa отвечает за анализ RuBQ / MKQA, mintaka - за импорт и фильтрацию Mintaka, 2wikimultihop - за обработку 2WikiMultiHopQA.

Отдельная часть финального проекта - слой merge slices. Он нужен для того, чтобы разные источники и срезы не оставались набором разрозненных JSON/CSV-файлов, а приводились к единому формату, совместимому с основным Wikidata-core. На вход merge-этапа поступают основной Wikidata-слой, обогащённые gold-ответы, выбранные внешние запросы, переведённые внешние примеры и служебные срезы. На выходе формируется единый merged_dataset, где каждый пример имеет общую схему полей и сохраняет информацию о своём происхождении.

Ключевой принцип объединения состоит в том, что merge не должен стирать различия между источниками. Для каждого примера сохраняются source_dataset, source_id или benchmark_id, тип среза, answerability, domain, complexity, query_text_ru, constraints, requested_count, gold_answer_qids, gold_answer_labels_ru и дополнительные служебные поля. Благодаря этому можно запускать инференс на единой таблице, но анализировать результаты отдельно по Wikidata-core, внешнему complex layer, zero-gold срезу, переводным запросам и другим подмножествам.

Такой unified format особенно важен для инфраструктурного запуска в Nirvana/YT. Модели и агенты получают одинаковый формат входа, а downstream-кубики оценки не зависят от того, из какого ноутбука пришёл пример. При этом аналитик может в любой момент отфильтровать результаты по source_dataset, типу reasoning, уровню сложности,

наличию околосинтетического критерия или *answerability*. Это делает бенчмарк не только коллекцией запросов, но и воспроизводимым экспериментальным контуром.

Формат записи поддерживает два уровня описания запроса. Естественнo-языковой уровень нужен для инференса LLM: модель получает *query_text_ru*. Формальный уровень нужен для проверки: *constraints*, *gold_answer_qids*, *gold_answer_labels_ru*, SPARQL, ASK validator и расширенные *gold aliases* после *enrichment*-этапа. Такое разделение позволяет проводить как автоматическую проверку, так и LLM-as-a-Judge оценку.

Ссылка на репозиторий: https://github.com/matveylogee/multihop_benchmark

2.8. Выводы по главе 2

Во второй главе была описана архитектура разработанного русскоязычного бенчмарка для оценки LLM на многоступенчатых и многокритериальных запросах. Главный вывод состоит в том, что для такой постановки недостаточно единого набора однотипных вопросов: разные классы запросов проверяют разные свойства модели и должны анализироваться отдельно. Из этого следует, что датасет нельзя строить как один общий набор вопросов без внутреннего деления. В работе поэтому используется несколько срезов: основной Wikidata-слой, простой контрольный набор, внешний слой сложных запросов, *zero-gold* примеры и отдельные запросы с редкими околосинтетическими условиями.

Центральное место в этой структуре занимает Wikidata-слой. Он выбран не потому, что Wikidata сама по себе является готовым бенчмарком, а потому что она даёт формальную основу для проверки: для каждого запроса можно сохранить ограничения, SPARQL-запрос, QID и русские названия эталонных ответов. Благодаря этому модель оценивается не только по совпадению с готовой строкой, но и по более тонким ошибкам: нарушению отдельного условия, неверному домену, правильному типу объекта при неправильном годе, стране, жанре или другом атрибуте.

В финальной версии этот слой был дополнительно усилен обогащением *gold*-ответов. *Enrichment* помогает учитывать разные варианты написания, русские и оригинальные названия, элиасы и частично неполные лейблы. За счёт этого проверка становится менее хрупкой: модель не штрафует только за другую форму записи правильной сущности, но при этом похожие, но неверные объекты не засчитываются автоматически.

Отдельным результатом проектирования стало введение среза запросов без корректного ответа. Такой срез необходим, потому что в реальном пользовательском сценарии модель не всегда должна выдавать список объектов: иногда правильное поведение состоит в отказе от ответа и объяснении, что заданные условия несовместимы. За счёт *near-miss* запросов без ответа бенчмарк позволяет выявлять галлюцинации, когда модель генерирует правдоподобные, но неверные объекты, а также ложные отказы, когда модель не пытается найти ответ в выполняемом запросе.

Также в главе показано, что существующие внешние датасеты нельзя механически объединять с основным корпусом. RuBQ и МКQA полезны как источники русских формулировок, но в основном содержат одношаговые вопросы. KQA Pro оказался наиболее полезным источником сложных логических паттернов, однако его запросы чаще являются англоязычными и одноответными. Mintaka и 2WikiMultiHopQA дают отдельные полезные примеры, но значительная часть их запросов относится к поверхностным bridge- или comparison-шаблонам. Поэтому внешние источники используются в работе не как замена Wikidata-слою, а как вспомогательный слой для расширения типов рассуждения и проверки отдельных сценариев.

Важной инженерной частью стала финальная сборка разных срезов в единую схему. Merge-этап позволяет запускать единый benchmark-пайплайн, не теряя происхождение и назначение каждого запроса. Благодаря этому результаты можно агрегировать по всему корпусу, но при интерпретации разделять Wikidata-core, external complex layer, zero-gold, simple control и другие подмножества. Именно это делает метрики содержательными: средний score не скрывает, на каком типе запросов модель ошибается.

Итогом главы является обоснование архитектуры бенчмарка как диагностического инструмента. Предложенное разделение на срезы позволяет получать не только общий скор модели, но и содержательную картину ошибок: где модель теряет условия, где ей не хватает поиска, где она галлюцинирует, где ошибается на сложном композиционном запросе, а где справляется только с простым контрольным срезом. Именно такая структура делает бенчмарк полезным для анализа реальных LLM-систем и дальнейшего сравнения моделей, а не просто коллекцией автоматически сгенерированных запросов.

ГЛАВА 3. ЭКСПЕРИМЕНТАЛЬНАЯ ВАЛИДАЦИЯ РЕШЕНИЯ

3.1. Валидация источников данных

Экспериментальная валидация в данной работе понимается прежде всего как проверка того, что бенчмарк можно построить воспроизводимо и что выбранные источники действительно соответствуют или не соответствуют целевой постановке. На этом этапе важен не только запуск моделей, но и доказательство того, что данные имеют правильную структуру, эталонные ответы проверяемы, а complexity labels отражают содержательную разницу запросов.

Первый результат валидации - подтверждение роли Wikidata как основного источника истины. В отличие от готовых QA-датасетов, Wikidata позволяет контролировать домены, свойства и ограничения, строить эталонные множества для списочных ответов и отличать answerable запросы от no-answer. Это особенно важно для многокритериальных запросов, где корректных ответов может быть много.

Второй результат анализа внешних источников оказался скорее отрицательным, но важным для проекта. Большинство готовых датасетов нельзя просто взять и добавить в основной корпус. RuBQ и МКQA хорошо помогают с русскими формулировками и простыми relation-паттернами, но для основного слоя они слишком одношаговые. В Mintaka много формально multi-hop примеров, однако значительная часть из них сводится к поверхностным bridge-запросам. 2WikiMultiHopQA, несмотря на большой объём, даёт много однотипных comparison и bridge-comparison шаблонов. Этот анализ подтверждает, что для целевой постановки нужен собственный основной Wikidata-слой, а внешние корпуса лучше использовать ограниченно: как доноры паттернов и вспомогательные диагностические срезы.

Третий результат - выделение KQA Pro как наиболее полезного внешнего источника. Он не решает проблему русского языка и ответов списком, но даёт сильный набор reasoning families. Поэтому его правильно использовать как вспомогательный сложный слой и банк паттернов для будущей генерации русскоязычных запросов.

3.2. Проектная структура итогового датасета

После анализа источников была зафиксирована итоговая структура датасета. Основу составляет Wikidata-слой: примерно 800 сложных запросов и около 200 простых контрольных примеров. Такой баланс нужен, чтобы сравнивать поведение моделей на двух режимах: базовом, где запросы ближе к обычному фактологическому QA, и сложном, где появляется несколько условий, списочный ответ и более высокая вероятность частичных ошибок. Внешний слой при этом занимает меньшую долю корпуса - он используется не как главная метрика, а как дополнительный диагностический материал.

Важный принцип - каждый пример должен иметь тип `answerability`. Для обычных запросов это `answerable`, для запросов без корректного ответа - `no_answer`. При этом срез запросов без корректного ответа не должен быть просто списком `failed examples`. Он должен содержать явно заданный `no_answer_type` и `no_answer_reason`. Иначе оценка галлюцинаций будет загрязнена техническими ошибками генератора.

Предлагаемая структура также позволяет проводить агрегированную и детальную аналитику. На верхнем уровне можно сравнивать модели по `overall скор`. На нижнем уровне можно анализировать качество по доменам, уровням сложности, числу условий, типу `reasoning`, наличию отрицания, полноте эталон и типу ответа.

Аналитический срез	Что показывает
<code>domain</code>	в каких предметных областях модель чаще ошибается
<code>complexity L1-L5</code>	как качество меняется при росте сложности
<code>number of constraints</code>	как влияет количество одновременных условий
<code>constraint type</code>	какие условия чаще игнорируются: <code>temporal</code> , <code>country</code> , <code>genre</code> , <code>rating</code> , <code>negation</code>
<code>answerability</code>	различие между обычными <code>answerable</code> запросами и <code>zero-gold</code>
<code>gold_truncated</code>	насколько метрики зависят от полноты <code>gold</code> -списка
<code>source split</code>	сравнение основного Wikidata-слоя и внешних вспомогательных запросов
<code>template_family</code>	поиск слабых шаблонов и однотипных ошибок

Для проверки структуры итогового датасета была построена статистика распределения запросов по доменам (рисунок 1). Она позволяет убедиться, что `benchmark` не сводится к одному тематическому направлению, а покрывает разные типы сущностей: фильмы, книги, географические объекты, страны, университеты, программное обеспечение, произведения искусства и другие домены. Такое распределение важно для диагностической ценности бенчмарка, поскольку разные домены проверяют разные виды фактических связей и разные типы ошибок моделей.

Статистика по доменам

	Домен	Запросов	Среднее_gold	Медиана_gold	Минимум_gold	Максимум_gold	Всего_gold_связей
2	cinema	191	48.25	9.00	2	618	9215
4	dishes	103	22.41	8.00	1	156	2308
3	countries	73	7.40	4.00	1	60	540
9	music_albums	73	54.53	14.00	4	480	3981
11	people	67	102.18	34.00	1	591	6846
18	videogames	65	50.42	10.00	1	592	3277
10	paintings	61	54.49	8.00	1	587	3324
8	museums	47	54.02	17.00	1	324	2539
12	physicists	47	81.19	15.00	1	663	3816
1	cars	45	45.49	14.00	5	768	2047
0	airports	42	131.83	44.00	2	667	5537
7	mathematicians	42	55.45	13.00	1	716	2329
5	geo	41	2.95	3.00	0	8	121
15	spacecraft	38	24.03	7.50	1	158	913
17	universities	35	144.23	54.00	9	762	5048
14	software	34	58.35	8.50	1	466	1984
13	smartphones	32	13.91	10.50	3	47	445
16	sports_clubs	30	125.70	37.00	5	607	3771
6	geo_ru	21	34.67	4.00	2	195	728

Рисунок 1 - Распределение запросов итогового датасета по доменам

Дополнительно было проанализировано распределение запросов по уровням сложности L1–L5 (рисунок 2). Такой анализ нужен для проверки того, что benchmark содержит как контрольные простые примеры, так и сложные multi-hop / multi-constraint запросы. Наличие сложных уровней позволяет отдельно анализировать деградацию качества моделей при увеличении числа условий и композиционной сложности запроса.

Статистика по уровням сложности

	Сложность	Запросов	Среднее_gold	Медиана_gold	Минимум_gold	Максимум_gold	Всего_gold_связей
0	L1	246	123.32	45.50	1	768	30337
1	L2	250	42.85	12.00	0	653	10712
2	L3	274	47.64	14.00	1	587	13053
3	L4	220	15.59	4.50	0	762	3430
4	L5	97	12.75	3.00	1	195	1237

Рисунок 2 - Распределение запросов по уровням сложности

Дополнительно была построена сводная статистика количества запросов по доменам и уровням сложности L1–L5 (рисунок 3). Эта таблица позволяет проверить, что итоговый датасет содержит не только общий набор запросов по разным предметным областям, но и внутреннее распределение сложности внутри каждого домена. Такой анализ важен, поскольку разные домены имеют разную структуру фактов и разную доступность эталонных ответов: например, для фильмов, блюд, стран, музеев или программного обеспечения используются разные типы ограничений и разные шаблоны multi-hop /

multi-constraint запросов. Наличие запросов нескольких уровней сложности в большинстве доменов позволяет проводить более детальную диагностику: сравнивать качество модели не только между доменами, но и при переходе от простых условий к более сложным композиционным запросам.

Количество запросов по доменам и уровням сложности

complexity	Домен	L1	L2	L3	L4	L5	Итого
2	cinema	41	30	20	66	34	191
4	dishes	16	21	31	20	15	103
3	countries	10	26	25	2	10	73
9	music_albums	15	16	25	15	2	73
11	people	15	17	25	10	0	67
18	videogames	21	19	16	9	0	65
10	paintings	10	12	17	11	11	61
8	museums	10	7	10	10	10	47
12	physicists	10	12	15	10	0	47
1	cars	8	12	15	10	0	45
0	airports	12	12	10	8	0	42
7	mathematicians	5	12	15	10	0	42
5	geo	20	9	7	5	0	41
15	spacecraft	5	6	6	9	12	38
17	universities	10	6	10	9	0	35
14	software	10	10	12	2	0	34
13	smartphones	8	8	9	6	1	32
16	sports_clubs	15	12	3	0	0	30
6	geo_ru	5	3	3	8	2	21

Рисунок 3 - Количество запросов по доменам и уровням сложности

Также была проанализирована средняя длина gold-list по доменам и уровням сложности (рисунок 4). Эта статистика показывает, насколько различается число эталонных ответов в разных частях датасета. Для бенчмарка это особенно важно, потому что запросы со списочным ответом могут иметь разную степень полноты: в одних случаях корректных ответов немного, и задача ближе к точному восстановлению одного или нескольких объектов, а в других случаях множество подходящих сущностей существенно больше, и на первый план выходят полнота ответа, recall и устойчивость к добавлению неподходящих объектов. Анализ средней длины gold-list помогает интерпретировать будущие метрики: низкий recall на доменах с большими gold-list может означать не полную ошибку модели, а неспособность покрыть всё множество корректных ответов, тогда как ошибка на доменах с малым gold-list чаще указывает на нарушение конкретных ограничений или подмену сущности.

Среднее число gold-ответов по доменам и уровням сложности

complexity	Домен	L1	L2	L3	L4	L5
0	airports	199.92	235.42	24.30	8.75	0.00
1	cars	131.88	21.17	11.93	55.90	0.00
2	cinema	113.32	8.13	162.05	10.42	11.65
3	countries	18.80	4.08	9.00	2.00	1.70
4	dishes	19.69	30.67	35.94	10.80	1.27
5	geo	3.60	2.11	2.57	2.40	0.00
6	geo_ru	6.80	25.33	68.67	2.75	195.00
7	mathematicians	322.00	22.25	26.53	5.40	0.00
8	museums	71.00	123.00	77.00	15.00	4.80
9	music_albums	99.67	22.00	73.72	18.33	8.00
10	paintings	80.10	28.25	126.12	1.91	1.73
11	people	284.07	76.88	47.56	8.90	0.00
12	physicists	320.60	17.42	23.80	4.40	0.00
13	smartphones	21.38	9.12	15.22	10.17	3.00
14	software	173.00	16.80	6.92	1.50	0.00
15	spacecraft	53.60	33.33	10.50	5.89	27.42
16	sports_clubs	225.13	27.83	20.00	0.00	0.00
17	universities	193.10	236.50	62.00	119.78	0.00
18	videogames	98.48	53.42	10.19	3.44	0.00

Рисунок 4 - Среднее число gold-ответов по доменам и уровням сложности

3.3. Инфраструктурный пайплайн инференса

После формирования JSON/JSONL-датасета начинается инфраструктурная часть эксперимента. Её назначение - провести один и тот же набор запросов через разные модели и агентные контуры, сохранить ответы в едином формате и автоматически посчитать метрики. Внутренний пайплайн строится вокруг инфраструктуры Yandex: данные хранятся в YT, запуски и преобразования организуются через Nirvana-графы, а промежуточные результаты агрегируются отдельными вычислительными кубиками.

YT в данном проекте выполняет роль единого хранилища запросов, ответов и оценок. Сгенерированный в ноутбуке JSON содержит естественно-языковый текст, constraints, QID эталонных ответов, русские названия эталонных ответов, domain, complexity, answerability, source_dataset и служебные метаданные. На первом этапе эти данные конвертируются в YT-таблицы, после чего каждый запрос получает стабильный benchmark_id, по которому можно связывать исходный пример, ответ модели, разметку судьи и агрегированные метрики.

Nirvana-граф обеспечивает воспроизводимость всего процесса. В нём явно фиксируются версии входных таблиц, параметры запуска, выбранные модели, промпты, режимы поиска и выходные таблицы. Это важно, потому что бенчмарк должен быть не разовым

ручным экспериментом, а повторяемым контуром оценки: при появлении новой модели или нового поискового агента можно прогнать те же срезы и сравнить не только общий score, но и профиль ошибок.

Отдельный инженерный слой занимают конверторы входных форматов. После ноутбука данные представлены в исследовательской схеме, удобной для анализа: `id`, `source_dataset`, `domain`, `query_text_ru`, `constraints`, `gold_answer_qids`, `gold_answer_labels`, `no_answer_type` и другие поля. Однако разные модели и агенты ожидают разные форматы запросов и системных инструкций. Поэтому перед инференсом данные приводятся к единому `inference-input` формату, где явно отделены текст запроса, служебные поля, `prompt template`, режим поиска и ожидаемый тип ответа.

После инференса ответы разных систем снова приводятся к общей схеме. Для каждого ответа сохраняются `benchmark_id`, вариант формулировки, модель, версия или алиас модели, режим поиска, `timestamp` запуска, сырой текст ответа, технический статус и ссылки на промежуточные артефакты. Такая таблица не является финальной оценкой: свободный ответ модели сначала проходит через модуль разметки, где из него извлекаются кандидатные сущности и проверяется соблюдение `criteria`.

В финальной архитектуре оценка разделена на два независимых шага. Первый шаг выполняет модель-судья: она разбирает ответ, извлекает сущности и размечает их по ограничениям. Второй шаг выполняет отдельный агрегирующий кубик `aggregate metrics`: он принимает структурированную разметку и по фиксированным формулам считает `precision`, `recall`, `hallucination rate`, `no-answer accuracy`, `constraint satisfaction` и итоговые `score` по срезам. Такое разделение уменьшает риск ошибки модели-судьи: она не должна сама придумывать итоговый балл, а только выполнить задачу, близкую к структурированному `fact-checking`.

3.4. Перефразирование, перевод и подготовка входов моделей

Часть запросов в основном Wikidata-слое рождается из формальных шаблонов. Это удобно для контроля `constraints`, но такие формулировки могут звучать слишком искусственно. Чтобы приблизить датасет к реальному пользовательскому языку, используется отдельный этап перефразирования. На вход модель получает один шаблонный русский запрос, а на выходе должна вернуть одну строку: живую, но семантически эквивалентную формулировку. Все ограничения - количество ответов, сущности, годы, страна, жанр, отрицания, редкие критерии - обязаны сохраниться без изменений.

Для перефразирования используется модель 32B 202506 FP8 Eagle3, то есть ускоренная версия YandexGPT Pro 5.1. Выбор такой модели обусловлен компромиссом между качеством редакторских перефразов и стоимостью массовой обработки. Перефразирование не должно превращаться в генерацию новых запросов: результат остаётся привязанным к тем же `constraints` и эталон-ответам. Поэтому промпт

сформулирован строго: запрещены подмена страны, жанра, диапазона лет, числа ответов и любого другого условия. Полный промпт перефразирования приведён в Приложении Б.

Для внешнего слой сложных запросов, прежде всего KQA Pro, добавляется этап перевода. Многие внешние источники исходно англоязычные, поэтому запрос нужно перевести на русский язык, не потеряв логическую структуру и сущности. Промпт перевода устроен аналогично промπτу перефразирования: модель не отвечает на вопрос, не упрощает его, не добавляет новых условий и не заменяет сущности более широкими категориями. В таблице сохраняются как оригинальный текст, так и русский перевод, чтобы при необходимости можно было восстановить происхождение запроса и проверить корректность перевода.

Важный принцип - разделение формулировки и истины. Перефразированный или переведённый текст может отличаться по синтаксису, порядку критериев и уровню вежливости, но формальный слой остаётся неизменным. Это позволяет сравнивать, как модели реагируют на разные естественные формулировки одного и того же запроса, не меняя эталон-набор и constraints.

3.5. Состав моделей для инференса

В экспериментальном контуре рассматриваются как одиночные LLM, так и агентные системы с поиском. Это принципиально важно, потому что разные слои бенчмарка проверяют разные навыки: параметрическую память, multi-hop reasoning, следование ограничениям, retrieval по редким фактам и устойчивость к провокациям без корректного ответа.

К основным внутренним системам относятся модели Алисы на Поиске, то есть генеративные ответы Алисы на поисковых сценариях. В этом случае речь идёт не об одной изолированной LLM, а об агенте, включающем несколько моделей, ретривер и логику обращения к поиску. Такой контур особенно важен для околосинтетических критериев и редких фактов: модель должна не только сформулировать ответ, но и найти релевантное свидетельство и не потерять constraints после retrieval-шага.

Отдельно рассматривается Нейроалиса с поиском на базе крупной модели. Она также интерпретируется как агентная система: ответ формируется не только языковой моделью, но и поисковым компонентом. Поэтому для неё важно измерять не только финальную текстовую корректность, но и способность соединить найденные факты с исходными условиями запроса.

В качестве внутренних baseline-моделей используются сильные LLM без обязательного доступа к поиску и reasoning-варианты, от которых ожидается более аккуратное пошаговое объединение условий. Такие модели нужны для проверки того, насколько далеко можно продвинуться только за счёт параметрической памяти и рассуждения без внешнего поиска. На простых запросах это часто достаточно, но на редких

идентификаторах, свежих фактах и узких пересечениях условий отсутствие retrieval становится отдельным источником ошибок.

Для внешнего сравнения могут запускаться актуальные публичные reasoning-модели с поиском. В оценочном контуре модель-судья вынесена отдельно: в финальной версии проекта она работает как API-модель ChatGPT 5.5 Thinking с поиском и используется не как тестируемая модель, а как инструмент структурированного разбора ответов. Для таких внешних сервисов важно фиксировать дату запуска, публичное название модели, режим поиска и параметры промпта, поскольку поведение публичных моделей может меняться со временем.

Итоговая таблица инференса должна хранить не только название модели, но и тип системы: параметрическая LLM, reasoning-модель, поисковый агент или внешняя модель с поиском. Такое разделение позволяет интерпретировать результаты корректно: плохое качество на околосинтетическом срезе у модели без поиска не является тем же типом ошибки, что плохое качество у поискового агента. В первом случае ожидаемым ограничением может быть отсутствие источника фактов, во втором - проблема маршрутизации поиска, извлечения evidence или объединения найденных данных.

3.6. Метрики, реворды и LLM-as-a-Judge

Оценка ответов моделей в данном бенчмарке строится как гибридная схема: с одной стороны, используются формальные эталонные ответы и автоматические метрики, с другой стороны, свободный текстовый ответ модели предварительно приводится к структурированному виду с помощью модели-судьи. Такая схема необходима, потому что тестируемая модель может отвечать не в виде чистого JSON-списка, а обычным текстом: перечислять объекты в предложении, добавлять пояснения, смешивать корректные и некорректные сущности, писать отказ или частично отвечать на запрос.

В финальной версии проекта модель-судья реализована как API-вызов ChatGPT 5.5 Thinking с поиском. Её задача принципиально отличается от задачи тестируемых моделей. Судья не должен сам решать benchmark-запрос и не должен самостоятельно выставить финальный score. Вместо этого он получает исходный запрос, constraints, answerability, gold-ответы или причину no-answer, а также сырой ответ проверяемой модели. После этого судья выполняет структурированный разбор: извлекает из ответа список кандидатных сущностей, нормализует их названия, сопоставляет с gold лейблами и элиасами, а затем проверяет каждую сущность по каждому критерию запроса.

Например, если пользователь просит назвать фильмы определённого жанра, периода, страны и рейтинга, а модель отвечает длинным текстом с пояснениями, судья сначала извлекает из него именно список фильмов. Затем для каждого фильма фиксируется, какие условия выполнены, а какие нарушены: жанр, год выхода, страна, рейтинг, участие актёра, отрицательное условие, редкий идентификатор или другой критерий.

Таким образом, результатом работы судьи является не одна субъективная оценка, а таблица атомарных фактов о каждом кандидатном объекте.

На выходе судья возвращает структурированную разметку: `extracted_entities`, `entity_judgments`, `matched_gold`, `is_correct`, `is_hallucination`, `satisfied_constraints`, `violated_constraints`, `possibly_correct_not_in_gold`, `missing_gold_examples`, `no_answer_refusal` и `no_answer_false_objects_listed`. Такая разметка позволяет отличить несколько типов ошибок, которые в обычной QA-метрике слились бы в один неправильный ответ: модель могла назвать реальный объект, но нарушить год; могла назвать объект правильного домена, но не того жанра; могла дать неполный список; могла выдумать несуществующую сущность; могла корректно отказаться на no-answer запросе; или, наоборот, отказаться там, где корректные ответы существуют.

Итоговые метрики считаются не судьёй, а отдельным агрегирующим кубиком `aggregate metrics` в Nirvana-графе. Это сделано намеренно: чем больше арифметики и глобальных решений принимает LLM-судья, тем выше риск нестабильности и труднообъяснимой ошибки. Поэтому модель отвечает только за `semantic parsing` и `fact-checking` ответа, а `precision`, `recall`, `F1`, `hallucination rate`, `no-answer accuracy`, `constraint satisfaction` и итоговые `score` вычисляются программно по фиксированным правилам.

$$Precision_i = |C_i| / |P_i|$$

где P_i - множество сущностей, извлечённых из ответа модели, а C_i - множество корректных сущностей, которые совпали с эталонами или были признаны удовлетворяющими `constraints` после проверки. Эта метрика показывает, какая доля перечисленных моделью объектов действительно корректна. Если модель перечисляет много правдоподобных, но неверных объектов, `precision` падает даже тогда, когда среди них есть один правильный ответ.

$$Recall_i = |C_i| / \min(|G_i|, k_i)$$

где G_i - эталонное множество, а k_i - число объектов, которое просили назвать в запросе. Использование $\min(|G_i|, k_i)$ важно, потому что пользователь часто просит не все возможные объекты, а фиксированное число примеров. `Recall` показывает, насколько хорошо модель покрыла требуемый список, а не просто нашла один релевантный объект.

$$ConstraintSat_i = (1 / |P_i|) * \sum_{p \in P_i} ConstraintSat(p)$$

Доля соблюдения ограничений нужна для анализа частично правильных ответов. Например, модель может назвать фильм нужного жанра и периода, но из другой страны; или программное обеспечение нужного типа, но не с требуемой лицензией. В таких случаях бинарная проверка «есть в gold / нет в gold» недостаточна: `constraint satisfaction` показывает, какое именно условие потеряно.

Для запросов без корректного ответа используется отдельная логика. Корректное поведение модели - не перечислять объекты, а явно сообщить, что подходящих ответов нет или что условия несовместимы. Поэтому aggregate metrics отдельно учитывает no_answer_refusal и no_answer_false_objects_listed. Модель получает высокую оценку на таком запросе только если она не галлюцинирует список и при этом распознаёт невозможность ответа.

Общий score модели рекомендуется анализировать не только как единую цифру, но и по срезам: Wikidata-core, easy control, external complex, zero-gold, semisynthetic/retrieval-required, отдельные домены и уровни сложности. Это принципиально важно для интерпретации: высокая средняя оценка может скрывать провал на no-answer запросах, а хорошее качество на популярных фильмах не означает устойчивости на редких околосинтетических критериях.

Таким образом, роль модели-судьи в работе состоит не в произвольном выставлении финального балла, а в разборе сложного свободного ответа и переводе его в структурированную форму. После этого оценка становится воспроизводимой: итоговые метрики считаются отдельным кодом по фиксированным формулам, а сама разметка остаётся интерпретируемой и пригодной для ручной проверки спорных случаев.

Метрика	Для каких запросов применяется	Интерпретация
precision@k	answerable list-style	доля корректных объектов среди выданных моделью
recall@k	answerable list-style с полным gold	покрытие gold-множества
constraint satisfaction	все answerable запросы	соблюдение каждого отдельного условия
hallucination rate	answerable и zero-gold	доля вымышленных или недопустимых объектов
no-answer accuracy	zero-gold	доля корректных отказов от ответа
judge score	сложные и неоднозначные ответы	интегральная экспертная оценка LLM-судьи
format adherence	все запросы	насколько ответ соответствует требуемому списку или отказу

3.7. Санити-чек, ручная валидация и ассессорская разметка

Автоматическая генерация и фильтрация не отменяют необходимости ручного sanity-check. В проекте часть слоёв запросов просматривалась автором вручную в роли ассессора. Такой ручной просмотр особенно важен для внешних датасетов и корзины

запросов без корректного ответа, где формальная метка `complex` или `empty-gold` сама по себе не гарантирует пригодность запроса для бенчмарка.

Ручная валидация использовалась на нескольких этапах. Для KQA Pro после автоматической фильтрации была сформирована таблица для ручной проверки, из которой отбирались действительно сложные и реконструируемые запросы. Для Mintaka и 2WikiMultiHopQA ручной просмотр помог отделить формальный `multi-hop` от поверхностных `bridge`-запросов, где требуется только перейти от одной сущности к одному простому атрибуту. Отдельный `sanity-check` применялся к основному Wikidata-слою. Проверялись русские `labels`, дубликаты эталонных ответов, соответствие текста запроса `constraints`, адекватность уровня сложности, отсутствие технических заглушек вроде `failed/skipped` и отсутствие случаев, когда эталон нарушает одно из условий. Для запросов без корректного ответа ручной просмотр особенно важен: нужно отличать настоящую логическую невозможность от ошибки генератора или неполноты источника данных.

После появления `enrichment`- и `merge`-этапов ручная проверка стала выполнять ещё одну функцию: контроль того, что обогащённые `aliases` действительно относятся к той же сущности, а объединение срезов не изменило смысл запроса. Проверялись случаи, где русский `label` мог быть неоднозначным, где оригинальное название совпадает у нескольких объектов, где внешний перевод меняет тип отношения, а также случаи, когда `source_dataset` или `answerability` могли быть перенесены неверно.

В рамках ассессорской разметки для сомнительных запросов фиксировались решения `keep/drop` и причина решения: слишком простой запрос, невалидный эталон, плохой русский `label`, неоднозначная формулировка, слишком очевидная провокация без корректного ответа, непроверяемый околосинтетический критерий или дубликат уже существующего паттерна. Такая разметка важна не только для очистки датасета, но и для будущего расширения: она показывает, какие автоматические фильтры стоит усилить.

Таким образом, валидация в проекте является гибридной: Wikidata и SPARQL дают формальную проверяемость, автоматические фильтры масштабируют отбор, `enrichment` улучшает сопоставление сущностей, `merge` обеспечивает единый формат, а ручная ассессорская проверка удаляет некачественные случаи, которые трудно надёжно отловить правилами. Для исследовательского бенчмарка это особенно важно, потому что плохой эталон или неудачная формулировка могут исказить выводы о качестве модели.

3.8. Ограничения, риски и направления развития

Первое ограничение связано с неполнотой Wikidata и внешних источников. Если факт отсутствует в графе или не имеет русского `label`, корректный с точки зрения реального мира ответ может не попасть в эталон. Поэтому в отчёте эталон трактуется как

проверяемый эталон относительно выбранного источника истины, а не как абсолютное описание мира. Enrichment-этап снижает часть проблем с названиями и aliases, но не может полностью исправить неполноту базы знаний.

Второе ограничение - качество околосинтетических критериев. Они полезны для проверки поискового поведения, но требуют аккуратной валидации. Если критерий невозможно проверить или он плохо представлен во внешних источниках, ошибка модели может быть вызвана не слабостью reasoning, а плохой постановкой запроса. Поэтому такие критерии нужно хранить отдельно и анализировать как retrieval-required срез.

Третье ограничение - недетерминированность агентных систем с поиском. Результат может зависеть от состояния поисковой выдачи, версии ретривера, актуальности документов и внутренней маршрутизации запроса. Чтобы уменьшить этот риск, необходимо сохранять метаданные запуска, версии моделей, дату, режим поиска и, по возможности, evidence или ссылки на найденные источники.

Четвёртое ограничение связано с LLM-as-a-Judge. Даже сильная reasoning-модель с поиском может ошибаться, особенно если ответ содержит редкие сущности, альтернативные названия или частично правильные списки. Поэтому судья используется как структуризатор и фактчекер ответа, а не как единственный источник финальной арифметики. Итоговые метрики считаются отдельным кодом, а спорные случаи должны оставаться доступными для ручной проверки.

Пятое ограничение связано с интерпретацией результатов, которые могут быть закрыты внутренними правилами доступа или NDA. В таких случаях бенчмарк всё равно полезен как исследовательский инструмент: даже без публикации численных score можно описывать классы ошибок, показывать обезличенные примеры, сравнивать поведение типов систем и фиксировать качественные выводы о том, какие сценарии являются наиболее проблемными.

Дальнейшее развитие проекта включает расширение доменов основного Wikidata-слоя, увеличение числа валидированных запросов без корректного ответа и околосинтетических запросов, улучшение сопоставления сущностей для русских названий и aliases, добавление оценки, учитывающей найденные свидетельства для поисковых агентов, а также накопление обезличенной базы типичных ошибок моделей для regression-тестирования.

3.9. Выводы по главе 3

В третьей главе была проведена экспериментальная валидация разработанного бенчмарка как инструмента анализа поведения LLM на сложных фактических запросах. Валидация показала, что предложенная структура датасета позволяет оценивать не только факт правильности или неправильности ответа, но и характер ошибки: потерю отдельного ограничения, неполноту списка, подмену сущности похожей,

галлюцинацию, ложный отказ от ответа или некорректное поведение на запросах без корректного ответа. Ключевое наблюдение состоит в том, что простые QA-запросы часто маскируют реальные ограничения моделей. На коротких одношаговых вопросах модель может выглядеть достаточно устойчивой, потому что ей нужно восстановить один популярный факт или одну связь. Однако при переходе к списочным multi-hop и multi-constraint запросам качество ответа становится существенно более хрупким. Модель должна одновременно удерживать тип объекта, домен, временной интервал, жанр, страну, рейтинг, отношение к другой сущности, отрицательное условие или дополнительный редкий критерий. Именно в таких сценариях становятся видны ошибки, которые не проявляются на стандартных фактологических бенчмарках. Качественный анализ ответов показал несколько типичных классов ошибок. Приведённые ниже примеры используются как качественные иллюстрации типичных ошибок и не раскрывают закрытые численные результаты внутренних прогонов.

1) Первый класс - пропуск части условий (рис. 5). Модель может выдать объекты правильного домена, например фильмы или книги, но не проверить один из фильтров: год выхода, страну, жанр, рейтинг или связь с указанным актёром, автором или организацией. Такой ответ внешне выглядит правдоподобно, но формально не удовлетворяет исходному запросу.



Рисунок 5 - Пример подмены условия в ответе НейроАлисы: критерий «произведены той же организацией, что и EnVision» заменён на более общий критерий принадлежности к ESA.

2) Второй класс - неполные списки (рис. 6): модель находит один или два корректных объекта, но не покрывает всё множество подходящих ответов или не достигает requested_count. Для обычного пользователя такой ответ может выглядеть полезным, однако для бенчмарка это важный сигнал потери recall.

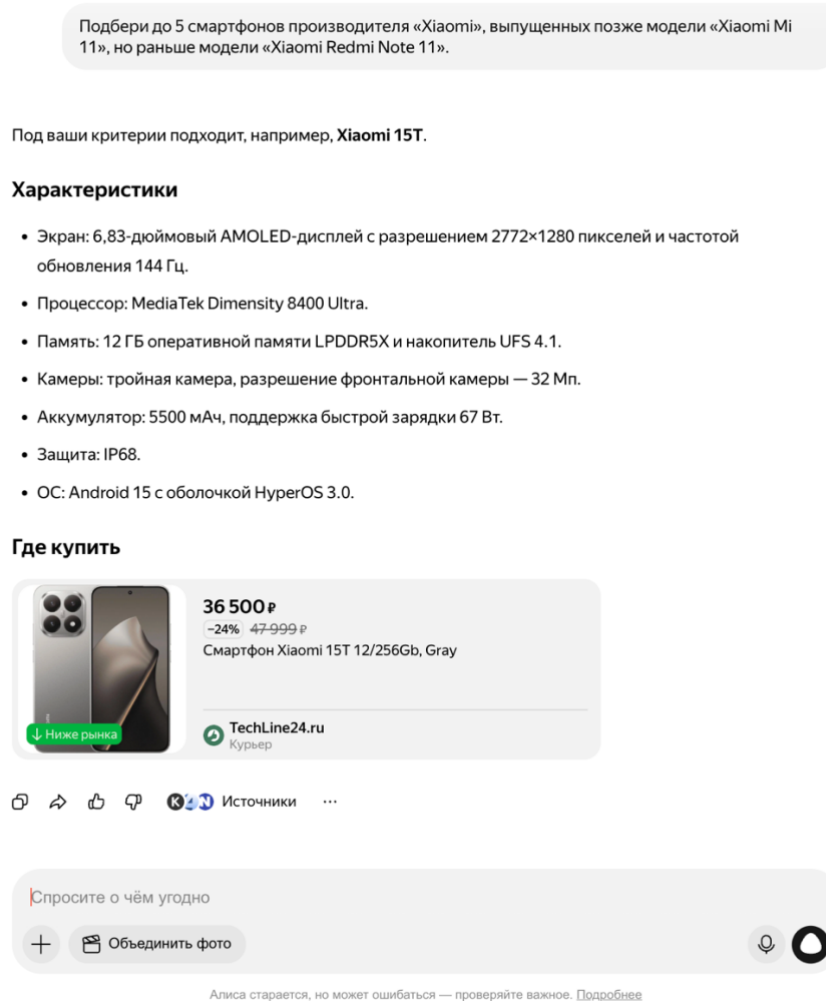


Рисунок 6 - Пример неполного списка: модель находит часть корректных объектов, но не покрывает всё множество подходящих gold-ответов.

3) Третий класс ошибок связан с подменой сущностей (рис. 7). В таких случаях модель называет объект, тематически близкий к правильному ответу, но не совпадающий с ним: фильм того же режиссёра, книгу того же автора, страну из того же региона, похожий географический объект или произведение из той же серии. Этот тип ошибок особенно важен для проверки reversal curse: модель может знать прямую ассоциацию между

сущностями, но при обращённом или многокритериальном запросе восстанавливает не точное множество ответов, а ближайшую по смыслу ассоциацию.

Назови 5 смартфонов производителя «Foxconn», выпущенных позже модели «Nokia 8110 4G».

Foxconn — крупнейший в мире контрактный производитель электроники, который занимается сборкой устройств по заказам других компаний. В открытых источниках нет информации о том, что Foxconn выпускала смартфоны под собственным брендом. Компания производила устройства для таких брендов, как Apple, Nokia, Xiaomi, OnePlus и других. [w.ru.wikipedia.org*+1](#)

Модель Nokia 8110 4G была выпущена в 2018 году. Если вас интересуют смартфоны, которые Foxconn собирала для других брендов после 2018 года, можно назвать, например, такие устройства:

1. **iPhone 12** (2020 год) — Foxconn участвовала в производстве этой модели смартфона Apple. [cnews.ru](#)
2. **iPhone 13** (2021 год) — компания также была одним из производителей этой модели. [cnews.ru](#)
3. **iPhone 14** (2022 год) — Foxconn продолжала сотрудничество с Apple в рамках производства новых поколений iPhone. [cnews.ru](#)
4. **iPhone 15** (2023 год) — модель, которую также собирали на фабриках Foxconn. [cnews.ru](#)
5. **iPhone 16** (2024 год) — последняя на данный момент модель iPhone, в производстве которой участвовала компания. [cnews.ru](#)

Также Foxconn участвовала в сборке смартфонов других брендов, например, Xiaomi и OnePlus, но конкретные модели в открытых источниках не упоминаются.

Если вам нужны дополнительные сведения о конкретных моделях или других аспектах деятельности Foxconn, уточните, пожалуйста, ваш запрос.

📄 🔗 📌 🗨️ 🌐 Источники ...

Спросите о чём угодно



📷 Изменить фото



Алиса старается, но может ошибаться — проверяйте важно. [Подробнее](#)

Рисунок 7 - Пример подмены сущности: модель называет тематически близкий объект, который не совпадает с эталонным ответом.

4) Четвёртый класс ошибок — галлюцинации (рис. 8). Они проявляются как в обычных сложных запросах, так и в zero-gold срезе. В обычных сложных запросах модель может добавить в список объект, который выглядит подходящим, но не проходит формальную проверку. В zero-gold срезе ошибка заметна ещё сильнее: корректный ответ должен состоять в том, что подходящих объектов нет, однако модель всё равно пытается собрать правдоподобный список. Поэтому zero-gold слой полезен не только как отдельный тип заданий, но и как проверка склонности модели уверенно отвечать там, где ответа быть не должно.

Перечисли 7 музыкантов и сольных исполнителей, которые связаны с жанром «современная христианская музыка», у которых есть ер и мини-альбомы этого жанра, с релизами в период 2015–2019 годов.

Вот семь музыкантов и сольных исполнителей, связанных с современной христианской музыкой, которые выпускали EP или мини-альбомы в период с 2015 по 2019 год:

1. **Hillsong Worship.** Австралийская группа, известная своими песнями для прославления и поклонения. В 2015 году выпустила EP *There Is More*. Также в 2018 году вышел EP *There Is More (Studio Sessions)*. [shazam.com +1](#)
2. **Слово жизни Music.** Российская группа. В 2019 году выпустила EP «Мы в Тебе одно». [ok.ru +1](#)
3. **Worship Culture Music.** В 2019 году выпустил альбом «Возвращение», который можно рассматривать как мини-альбом, так как он содержит несколько треков. [deezer.com](#)
4. **Wolrus Worship.** Группа, работающая в жанре христианской музыки. В 2019 году выпустила альбом «Орлы (Live)», который по структуре близок к мини-альбому. [ok.ru](#)
5. **DZIANIS.** Исполнитель в жанре христианской музыки и госпела. В 2019 году участвовал в выпуске EP *DZIANIS (Remixes)* совместно с Wolrus Worship. [music.apple.com +2](#)
6. **HILLSONG WORSHIP RUSSIA.** В 2019 году выпустил альбом «Я не одинок», который по формату можно отнести к мини-альбому. [music.yandex.ru](#)
7. **Лев.** Сольный исполнитель, связанный с «Словом жизни Music». В 2019 году выпустил EP «Мы в Тебе одно» в составе группы. [music.apple.com](#)

Обратите внимание, что классификация релизов как EP или мини-альбомов иногда может быть субъективной и зависит от количества треков и формата выпуска. Для уточнения деталей рекомендуется проверять информацию на официальных ресурсах исполнителей или в музыкальных базах данных.

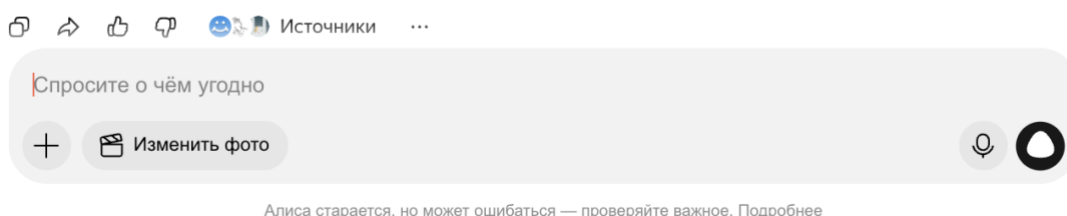


Рисунок 8 - Пример галлюцинации: в ответ добавлен объект, который не подтверждается формальным слоем проверки или не удовлетворяет условиям запроса.

5) Пятый класс ошибок - ложные отказы (рис. 9). В таких случаях модель не пытается восстановить список сущностей и отвечает в духе «не удалось найти информацию» или «данных недостаточно», хотя корректные ответы существуют. Это не галлюцинация, но для ассистента такая ошибка тоже существенна: система оказывается бесполезной там, где могла бы решить задачу через поиск, параметрическую память или пошаговое сопоставление условий. На практике это особенно важно для пользовательских ассистентов, потому что чрезмерно частые отказы снижают полезность системы даже при отсутствии явных фактических ошибок.

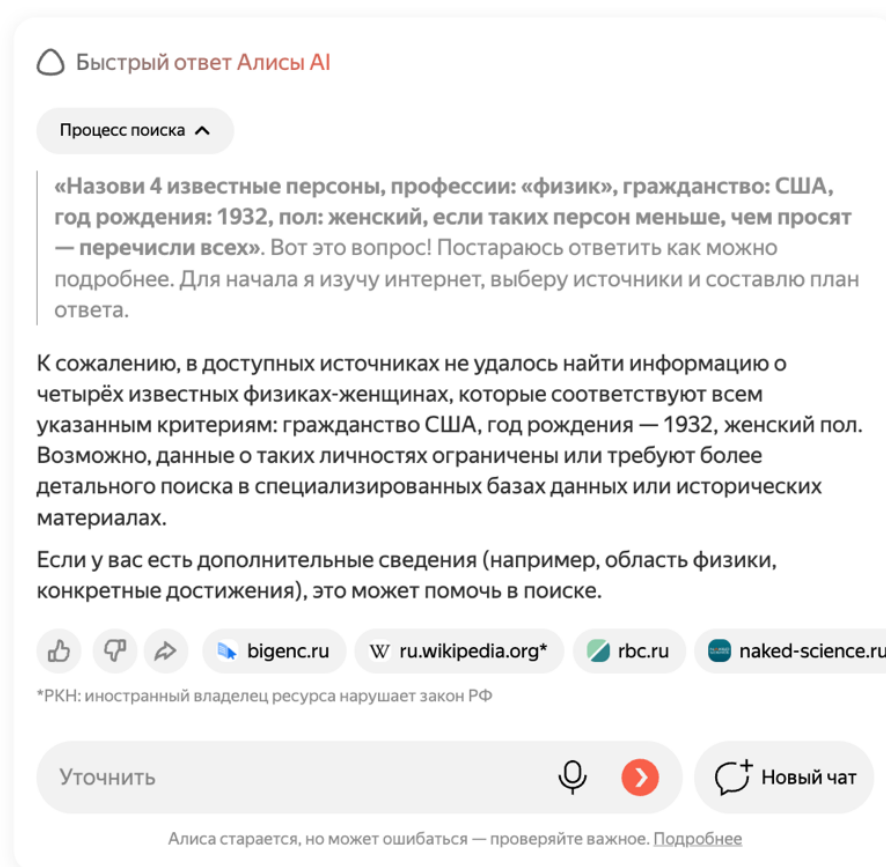


Рисунок 9 - Пример ложного отказа: модель отказывается от ответа или сообщает о недостатке информации, хотя корректные ответы существуют.

Сравнение поведения разных типов моделей показывает, что модели без поиска чаще полагаются на параметрическую память и поэтому хуже справляются с редкими, околосинтетическими и многокритериальными условиями. Они либо отказываются от ответа, либо выдают популярные объекты из нужного домена без строгой проверки всех критериев. Более сильные reasoning-модели лучше удерживают структуру запроса и чаще пытаются разложить его на условия, но и у них остаются ошибки неполного покрытия gold-ответов и потери отдельных constraints. Поисковые и retrieval-augmented системы в среднем лучше работают с редкими фактами, однако наличие поиска само по себе не решает проблему: после retrieval-шага модель всё ещё должна правильно извлечь сущности, сопоставить найденные факты и проверить все ограничения одновременно. Отдельно была проверена полезность LLM-as-a-Judge контура. В финальной схеме модель-судья не отвечает за итоговый балл целиком. Её роль более узкая: разобрать свободный ответ, выделить кандидатные сущности, привести их к нормализованной форме, сопоставить с gold labels и aliases и проверить отдельные критерии запроса. После этого она возвращает атомарную разметку, а не готовый precision или recall. Итоговые метрики — precision, recall, constraint satisfaction, hallucination rate, no-answer accuracy и общий reward — считаются уже отдельным модулем aggregate metrics. Такое

разделение оказалось важным: модель помогает с семантическим разбором, а арифметика и агрегация остаются детерминированными.

Таким образом, экспериментальная валидация подтвердила главную идею работы: бенчмарк полезен не только как таблица вопросов для сравнения моделей по одному итоговому числу. Он показывает, из-за чего именно ломается ответ. Модель может не найти нужный факт, потерять одно из условий, перепутать близкие сущности, добавить неподтверждённый объект, отказаться при наличии ответа или неверно обработать пустое множество. Такая детализация особенно важна для реальных LLM-систем и поисковых ассистентов: в них опасны не только уверенные неверные ответы, но и чрезмерная осторожность, когда модель без необходимости уходит в отказ.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы был разработан русскоязычный исследовательский бенчмарк для оценки эффекта reversal curse и связанных с ним ошибок LLM в сложных multi-hop и многокритериальных запросах. Основная задача работы состояла не в обучении новой модели, а в создании воспроизводимого измерительного инструмента, который позволяет проверять, насколько LLM и поисковые ассистенты способны отвечать на фактические запросы со списочными ответами, несколькими ограничениями, обращёнными отношениями и случаями отсутствия корректного ответа.

В работе была предложена многоуровневая архитектура датасета. Центральным компонентом стал основной Wikidata-слой, в котором для каждого запроса сохраняются естественно-языковая формулировка на русском языке, формальные constraints, SPARQL-запрос, QID эталонных ответов, русские **лейблы** и дополнительные служебные поля для проверки. Такой формальный слой источника истины позволяет отделить ошибку модели от неоднозначности вопроса и делает бенчмарк воспроизводимым.

Дополнительно были проанализированы внешние источники данных: RuBQ, MKQA, KQA Pro, Mintaka и 2WikiMultiHopQA. Анализ показал, что ни один из этих датасетов не может напрямую заменить основной Wikidata-слой: одни источники слишком просты и одношаговы, другие англоязычны, третьи в основном содержат одноответные или однотипные bridge/comparison-запросы. Поэтому внешние источники были использованы как вспомогательные слои и доноры логических паттернов, а не как механически объединённая замена основного корпуса.

В финальной версии проекта пайплайн был расширен несколькими важными инженерными компонентами. Был добавлен этап обогащения gold-ответов, который повышает устойчивость оценки за счёт русских labels, aliases, альтернативных названий и вспомогательных признаков для нормализации ответов моделей. Также был реализован этап объединения разных срезов датасета в единую Wikidata-like схему с сохранением происхождения каждого запроса, answerability, constraints и gold-полей. Это позволяет запускать общий контур инференса и оценки, но при этом анализировать каждый слой отдельно.

Отдельным результатом стало проектирование среза запросов без корректного ответа. Такой слой необходим для проверки того, умеет ли модель не только находить ответы, но и корректно отказываться от них при несовместимых условиях. В отличие от очевидных искусственных примеров, в работе используются near-miss запросы, которые выглядят правдоподобно, но имеют скрытое противоречие. Это делает zero-gold слой полезным инструментом для диагностики галлюцинаций и чрезмерной уверенности модели.

Для оценки ответов была описана схема, в которой разбор ответа и подсчёт метрик разведены по разным этапам. LLM-as-a-Judge используется не как «чёрный ящик», выставляющий финальную оценку, а как инструмент структурирования ответа: он извлекает кандидатные сущности, нормализует их, сопоставляет с эталонами и aliases, проверяет отдельные критерии и фиксирует типы ошибок. После этого агрегированные метрики и reward считаются отдельным модулем aggregate metrics. Такой подход делает оценку более проверяемой: модель-судья помогает там, где нужна языковая интерпретация, но итоговый score не зависит от её произвольного суммарного суждения. Проведённая экспериментальная валидация показала, что разработанный бенчмарк действительно подсвечивает ошибки, которые плохо заметны на обычных одношаговых QA-наборах. Он позволяет отдельно увидеть пропуск условий, неполные списки, подмену похожими сущностями, неустойчивость при обращении фактических связей, галлюцинации в сложных запросах и ошибки на no-answer примерах. Для работы это принципиально: результатом становится не только общий показатель качества, но и объяснимая картина того, почему конкретный ответ оказался неправильным.

Практическая ценность бенчмарка состоит в том, что его можно использовать как диагностический инструмент для русскоязычных LLM, поисковых ассистентов и retrieval-augmented систем. Он позволяет сравнивать модели не только по доле правильных ответов, но и по характеру ошибок: какие ограничения чаще теряются, где модель начинает выдумывать сущности, в каких случаях отказывается от ответа и насколько хорошо она объединяет найденные факты в один связный результат.

В результате цель работы можно считать достигнутой. Был собран воспроизводимый пайплайн генерации, обогащения, объединения и оценки запросов; построен формальный truth layer на основе Wikidata; проанализированы внешние источники сложных QA-паттернов; выделен zero-gold слой для проверки галлюцинаций; описана схема LLM-as-a-Judge и aggregate metrics. Получившийся бенчмарк может быть использован как основа для дальнейших экспериментов: расширения числа доменов, добавления новых типов ограничений и более детального сравнения современных LLM-систем на русскоязычных multi-hop и многокритериальных запросах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, Owain Evans. The Reversal Curse: LLMs trained on “A is B” fail to learn “B is A” // arXiv.org. 2023. URL: <https://arxiv.org/abs/2309.12288>
2. Denny Vrandečić, Markus Krötzsch. Wikidata: A Free Collaborative Knowledge Base // Communications of the ACM. 2014. Vol. 57, No. 10. P. 78-85. DOI: 10.1145/2629489.
3. Ivan Rybin, Vladislav Korablinov, Valentin Biryukov, Mikhail Burtsev. RuBQ 2.0: An Innovated Russian Question Answering Dataset // Communications in Computer and Information Science. 2021. DOI: 10.1007/978-3-030-77385-4_32.
4. Shayne Longpre, Yi Lu, Joachim Daiber. MKQA: A Linguistically Diverse Benchmark for Multilingual Open Domain Question Answering // Transactions of the Association for Computational Linguistics. 2021. URL: <https://aclanthology.org/2021.tacl-1.82/>
5. Shulin Cao, Jiaxin Shi, Liangming Pan, Lunyu Nie, Yutong Xiang, Lei Hou, Juanzi Li, Bin He, Hanwang Zhang. KQA Pro: A Dataset with Explicit Compositional Programs for Complex Question Answering over Knowledge Base // ACL. 2022. URL: <https://aclanthology.org/2022.acl-long.422/>
6. Priyanka Sen, Alham Fikri Aji, Amir Saffari. Mintaka: A Complex, Natural, and Multilingual Dataset for End-to-End Question Answering // COLING. 2022. URL: <https://aclanthology.org/2022.coling-1.138/>
7. Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, Akiko Aizawa. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps // COLING. 2020. URL: <https://aclanthology.org/2020.coling-main.580/>
8. Jiawei Gu, Xuhui Jiang, Zhichao Shi et al. A Survey on LLM-as-a-Judge // arXiv.org. 2024. URL: <https://arxiv.org/abs/2411.15594>
9. Haitao Li, Qian Dong, Junjie Chen et al. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods // arXiv.org. 2024. URL: <https://arxiv.org/abs/2412.05579>
10. Alon Talmor, Jonathan Berant. The Web as a Knowledge-base for Answering Complex Questions // NAACL. 2018. URL: <https://aclanthology.org/N18-1059/>
11. Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, Christopher D. Manning. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering // EMNLP. 2018. URL: <https://arxiv.org/abs/1809.09600>
12. Vladislav Korablinov, Pavel Braslavski. RuBQ: A Russian Dataset for Question Answering over Wikidata // ISWC. 2020. URL: <https://arxiv.org/abs/2005.10659>
13. matveylogee. multihop_benchmark: русскоязычный исследовательский бенчмарк для оценки multi-hop и многокритериальных запросов // GitHub repository. 2026. URL: https://github.com/matveylogee/multihop_benchmark

14. Wikidata community. Wikidata Query Service and Wikidata documentation // Wikidata.org. URL: <https://www.wikidata.org/>

ПРИЛОЖЕНИЕ А. СТРУКТУРА ОДНОЙ ЗАПИСИ БЕНЧМАРКА

Ниже приведён рекомендуемый состав полей одной записи датасета. Структура поддерживает одновременно естественно-языковое описание запросы и формальный слой проверки.

id - уникальный идентификатор примера; domain - домен запроса; complexity - уровень сложности L1-L5; query_text_ru - русский текст пользовательского запроса; constraints - сериализованный набор ограничений; requested_count - число элементов, которое должна вернуть модель; gold_answer_qids - список QID эталонных ответов; gold_answer_labels_ru - русские названия эталонных сущностей; sparql_query - исходный SELECT-запрос; gold_truncated - флаг усечения эталонного списка; ask_validator_sparql - ASK-шаблон для проверки candidate-ответа; answerability - тип запроса answerable/no_answer; no_answer_type и no_answer_reason - поля для запросов без корректного ответа.

Иллюстративный пример: запрос «Назови 5 фильмов жанра “криминальный фильм” из США, 2015 года» хранится не только как строка query_text_ru, но и как constraints: kind=film, genre=crime film, country=USA, year_from=2015, year_to=2015, requested_count=5. Gold-ответы хранятся как QID и русские labels, а SPARQL/ASK позволяют формально проверить кандидатов.

ПРИЛОЖЕНИЕ Б. ПРОМПТЫ ПЕРЕФРАЗИРОВАНИЯ, ПЕРЕВОДА И LLM-AS-A-JUDGE

Б.1. Промпт перефразирования русскоязычных запросов

Ты — редактор запросов. Твоя задача — перефразировать пользовательский запрос на русском так, чтобы он звучал как от живого человека, а не шаблонно.

ВХОД: один исходный запрос (обычно шаблонный).

ЦЕЛЬ: сделать формулировки разнообразнее (лексика, порядок условий, синтаксис, вопрос/просьба, вежливость), но сохранить тот же смысл и те же ограничения.

ВЫХОД: одна строка, перефразированный запрос на русском.

ЖЁСТКИЕ ПРАВИЛА (обязательные):

1. НЕ меняй смысл запроса и НЕ добавляй новых условий.
2. НЕ подменяй сущности на другие.
 - Страна должна остаться той же самой (например, Франция не может стать Германией).
 - Жанр/тип/класс должен остаться тем же (если указан «документальный фильм», это не должно стать «комедия» и т.п.).
 - Диапазон лет и все числа должны совпадать точно (2010–2019 ≠ 2011–2020; 5 заменить на "пять" — можно).
3. Разрешена ТОЛЬКО «естественная» лексика для тех же сущностей:
 - можно склонять и менять форму: «Франция» → «из Франции», «французские», «во Франции» (если смысл остаётся «страна происхождения = Франция»).
 - можно перефразировать жанр, если это ТОЧНО то же самое: «жанра “документальный фильм”» → «документальных фильмов» / «в жанре “документальный фильм”».
 - нельзя заменять сущность синонимом, который может расширить/сузить смысл (например, «Франция» → «Европа» нельзя).
4. Если в исходнике есть кавычки «...» вокруг сущности — старайся сохранить саму сущность, но можно менять конструкцию вокруг неё.
5. Не используй служебные метки, не пиши объяснений, не добавляй списки «критериев». Только текст перефраз.
6. В перефразированном запросе должны явно присутствовать: количество, сущности, временной диапазон (если он был), и все остальные ограничения.
7. Нельзя подменять критерий страны на язык/локацию/производство и т.п.

ТРЕБОВАНИЯ К РАЗНООБРАЗИЮ:

- нейтральный, короткий (может быть вопросом).
 - другой порядок критериев (например, годы → страна → жанр).
 - более разговорный/вежливый (“подскажи/пожалуйста/хочу посмотреть”), но без лишней воды.
- Старайся варьировать лексику/порядок условий/тон.

ПРИМЕРЫ ПЕРЕФРАЗИРОВАНИЯ

Исходный:

Назови 5 фильмов, жанра «документальный фильм», из страны: Франция, в период 2010–2019 годов.

Корректные примеры:

Можешь перечислить 5 документальных фильмов из Франции за период 2010–2019 годов?

Подбери 5 фильмов за 2010–2019 годы из страны: Франция, жанра «документальный фильм».

Хочу посмотреть что-нибудь французское за 2010–2019: посоветуй 5 фильмов в жанре «документальный фильм».

Некорректные примеры:

Посоветуй 5 фильмов из Германии за 2010–2019. (страна подменена)

Назови 7 фильмов жанра «документальный фильм» из Франции. (число изменено)

Подбери фильмы из Франции в 2010–2020 годах. (годы изменены)

Теперь перефразируй следующий запрос.

Б.2. Промпт перевода внешних запросов на русский язык

Ты — переводчик и редактор benchmark-запросов для оценки LLM.

ВХОД: один исходный запрос на английском языке. Запрос может содержать несколько условий, сущности, даты, числа, сравнения, отрицания, ограничения по времени или редкие идентификаторы.

ЦЕЛЬ: перевести запрос на русский язык так, чтобы он звучал естественно для русскоязычного пользователя, но полностью сохранял смысл, все сущности и все ограничения.

ВЫХОД: одна строка на русском языке. Не отвечай на запрос и не добавляй пояснений.

ЖЁСТКИЕ ПРАВИЛА:

1. Не решай запрос. Нужно только перевести формулировку.
2. Не добавляй новых условий и не удаляй существующие.
3. Не заменяй сущности на похожие или более общие:
 - название фильма, книги, страны, города, организации или человека должно остаться тем же объектом;
 - если имя обычно не переводится, оставь его в оригинальной форме;
 - если существует общеупотребительный русский вариант названия, можно использовать его.
4. Все числа, годы, диапазоны дат, порядковые номера и сравнения должны сохраниться точно.
5. Отрицания, кванторы и логические связи должны сохраниться:
 - not / except / without / no → «не», «кроме», «без», «ни одного» по смыслу;
 - before / after / earlier / later → корректные временные отношения;
 - at least / at most / more than / fewer than → точные количественные ограничения.
6. Не подменяй тип отношения:
 - country of origin ≠ language;
 - located in ≠ produced in;
 - author ≠ director;
 - cast member ≠ character.
7. Если запрос звучит слишком технически, можно сделать русский вариант чуть более естественным, но нельзя менять constraints.
8. Верни только итоговый русский запрос одной строкой.

Теперь переведи следующий запрос.

Б.3. Промпт LLM-as-a-Judge

Ты — строгий ассессор, структуризатор и фактчекер ответов LLM на русскоязычные многошаговые и многокритериальные запросы.

Твоя задача — НЕ выставить финальный score модели, а привести свободный ответ модели к структурированному виду, чтобы затем метрики были посчитаны программно отдельным aggregate metrics модулем по фиксированным формулам.

Тебе дано:

1. user_query — исходный пользовательский запрос;
2. answerability — answerable или no_answer;

3. `constraints` — список обязательных условий запроса;
4. `requested_count` — сколько объектов просили назвать;
5. `gold_labels` и `gold_aliases` — эталонные ответы и варианты написания из источника истины, если запрос `answerable`;
6. `no_answer_reason` — причина отсутствия корректного ответа, если запрос `no_answer`;
7. `model_answer` — ответ проверяемой модели.

Что нужно сделать:

- извлечь из `model_answer` все явно названные сущности-кандидаты, игнорируя служебный текст, рассуждения, объяснения и общие фразы;
- нормализовать названия кандидатов: учесть русские названия, оригинальные названия, `aliases`, транслитерации и очевидные варианты написания;
- сопоставить кандидатов с `gold_labels` / `gold_aliases`, если это та же сущность;
- для каждой сущности проверить выполнение каждого ограничения из `constraints`;
- отметить корректные сущности, галлюцинации, неподдержанные ответы и частично корректные ответы;
- для `no_answer` запросов проверить, был ли корректный отказ и перечисляла ли модель ложные объекты;
- дать краткое объяснение разметки.

Жёсткие правила:

1. Не засчитывай объект, если он нарушает хотя бы одно обязательное условие запроса.
2. Не засчитывай объект только потому, что он похож по теме или звучит правдоподобно.
3. Если объект отсутствует в `gold_labels`, но кажется потенциально корректным, помести его в `possibly_correct_not_in_gold` и объясни причину.
4. Если `answerability = no_answer`, лучший ответ — явное сообщение, что подходящих объектов нет или условия несовместимы. Перечисление объектов в таком случае является ошибкой.
5. Не штрафуй за стиль, если фактическая часть корректна.
6. Штрафуй за изменение условий: другой год, другая страна, другой жанр, игнорирование отрицания, игнорирование редкого или околосинтетического критерия.
7. Не считай итоговые `precision`, `recall`, `F1` и `ModelScore` самостоятельно. Верни атомарную разметку; метрики будут рассчитаны отдельным кодом.

Верни строго JSON без markdown:

```
{
  "extracted_entities": [...],
  "entity_judgments": [
    {
      "entity": "...",
      "normalized_entity": "...",
      "matched_gold": "... или null",
      "is_correct": true,
      "is_hallucination": false,
      "satisfied_constraints": [...],
      "violated_constraints": [...],
      "comment": "краткое объяснение"
    }
  ],
  "correct_entities": [...],
  "hallucinated_entities": [...],
  "possibly_correct_not_in_gold": [...],
  "missing_gold_examples": [...],
  "no_answer_refusal": true,
  "no_answer_false_objects_listed": false,
  "short_explanation": "краткое объяснение разметки"
}
```

ПРИЛОЖЕНИЕ В. СВЯЗЬ ГИПОТЕЗЫ С МЕТРИКАМИ И РЕВОРДАМИ

Исследовательская гипотеза работы связывает архитектуру бенчмарка с измеримостью поведения моделей. Если запросы имеют формальный слой проверки, а датасет разделён на основной Wikidata-слой, внешний слой сложных запросов, срез запросов без корректного ответа и околосинтетические критерии, то качество LLM-систем можно анализировать не одной общей оценкой, а набором интерпретируемых метрик.

Для обычного запроса с корректными ответами используются следующие обозначения: G_i - множество эталонных ответов, P_i - множество сущностей, извлечённых моделью-судьёй из ответа модели, C_i - множество корректных сущностей, H_i - множество галлюцинированных или неподдержанных сущностей, k_i - запрошенное число ответов.

$$Precision_i = |C_i| / |P_i|$$

$$Recall_i = |C_i| / \min(|G_i|, k_i)$$

$$F1_i = 2 * Precision_i * Recall_i / (Precision_i + Recall_i)$$

$$HallucinationRate_i = |H_i| / |P_i|$$

Для проверки следования условиям используется метрика $ConstraintSat_i$. Она считается как средняя доля выполненных ограничений по всем извлечённым сущностям. Если в запросе есть m_i условий, а для сущности p выполнено r_p условий, то $ConstraintSat(p) = r_p / m_i$, а $ConstraintSat_i$ - среднее значение $ConstraintSat(p)$ по всем p из P_i .

Итоговая автоматическая оценка обычного запроса считается по формуле:

$$AutoScore_i = 0.45 * F1_i + 0.30 * ConstraintSat_i + 0.25 * (1 - HallucinationRate_i)$$

Для запросов без корректного ответа используется отдельная оценка. $Refusal_i$ равен 1, если модель корректно отказалась от перечисления объектов, и 0 иначе. $NoObject_i$ равен 1, если модель не перечислила ложные объекты, и 0 иначе.

$$ZeroScore_i = 0.70 * Refusal_i + 0.30 * NoObject_i$$

Общий скор отдельного запроса выбирается по типу запроса:

$$QueryScore_i = AutoScore_i \text{ для answerable-запросов; } QueryScore_i = ZeroScore_i \text{ для запросов без корректного ответа.}$$

Для отдельного среза датасета с итоговая оценка считается как среднее по всем запросам этого среза:

$$Score_s = (1 / N_s) * \sum_{\{i \in s\}} QueryScore_i$$

Общий скор модели на бенчмарке может считаться как взвешенное среднее по срезам:

$$ModelScore = \sum_s w_s * Score_s, где \sum_s w_s = 1$$

Модель-судья используется не для произвольного выставления финального балла, а для структурирования свободного ответа и разметки атомарных решений: какие сущности названы, какие из них корректны, какие ограничения нарушены и есть ли галлюцинации. После этого метрики считаются программно по фиксированным правилам. Такой подход сохраняет интерпретируемость результата и позволяет отдельно анализировать разные типы ошибок моделей.

Часть гипотезы	Как проверяется
Модель теряет часть условий в сложных запросах	constraint satisfaction по каждому условию и сравнение L1-L5
Стандартные QA-датасеты недостаточно выявляют ошибки	сравнение свойств RuBQ/MKQA/Mintaka/2Wiki с основным Wikidata-слоем
Внешний слой сложных запросов полезен как донор паттернов	наличие KQA Pro reasoning families и ручная проверка отобранных 62 примеров
Запросы без корректного ответа выявляют галлюцинации	no-answer accuracy, hallucination rate и разметка корректного отказа
Многоуровневый бенчмарк диагностичнее единого смешанного датасета	отдельная аналитика по source_split, answerability, complexity и domain